SDU

Software Architecture (T630019402)

Lecture #9, April 21, 2026

Jukka Ruohonen

Case #8

- Sketch a faulty distributed system with which three AI agents collaborate over a network. One agent should analyze code and write bug reports; another agent should do bug fixes and make pull requests; and the third agent should approve and merge the requests. The emphasis is on the adjective *faulty*; the sketch should demonstrate at least three *architectural* security weaknesses and two other *architectural* weaknesses related to other quality attributes addressed thus far in the course. You can elaborate the weaknesses in writing (and orally in the class), with diagrams, or both.

Survey Status Update

- 11 responses thus far (17 April)
- Expected grades: 4 ($n = 1$), 7 ($n = 6$), and 10 ($n = 4$)
- Attended lectures $\in [2, 9]$ with a median of 3.5
- $Cor(\text{expected grades}, \text{attended lectures}) \simeq 0.83$
- Good points regarding the MCQ plan, thanks!

A Few Short Mock-up Questions Again (1/7)

What is the Difference Between Fail-Safe and Fail-Fast?

A Few Short Mock-up Questions Again (2/7)

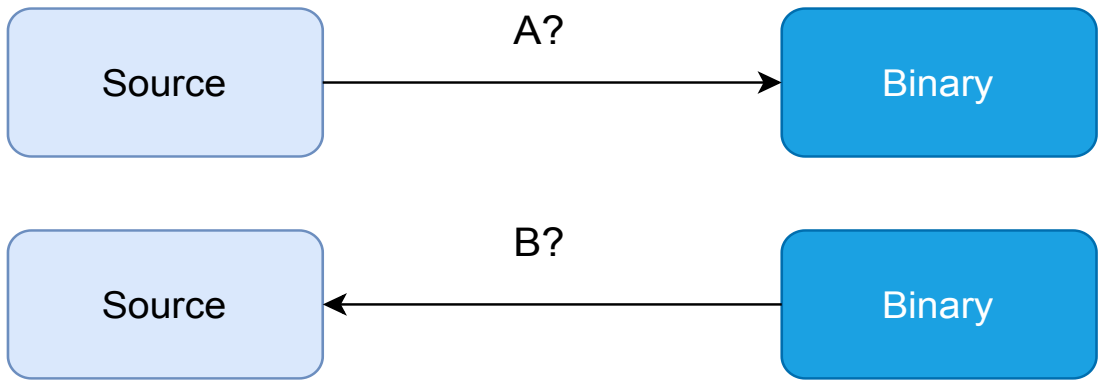


Figure 1: (a) What the Figure Might Denote, and (b) What Concepts A and B Might Be for Making it Explicit?

A Few Short Mock-up Questions Again (3/7)

What Are Code Sanitizers and Why They Are Useful?

A Few Short Mock-up Questions Again (4/7)

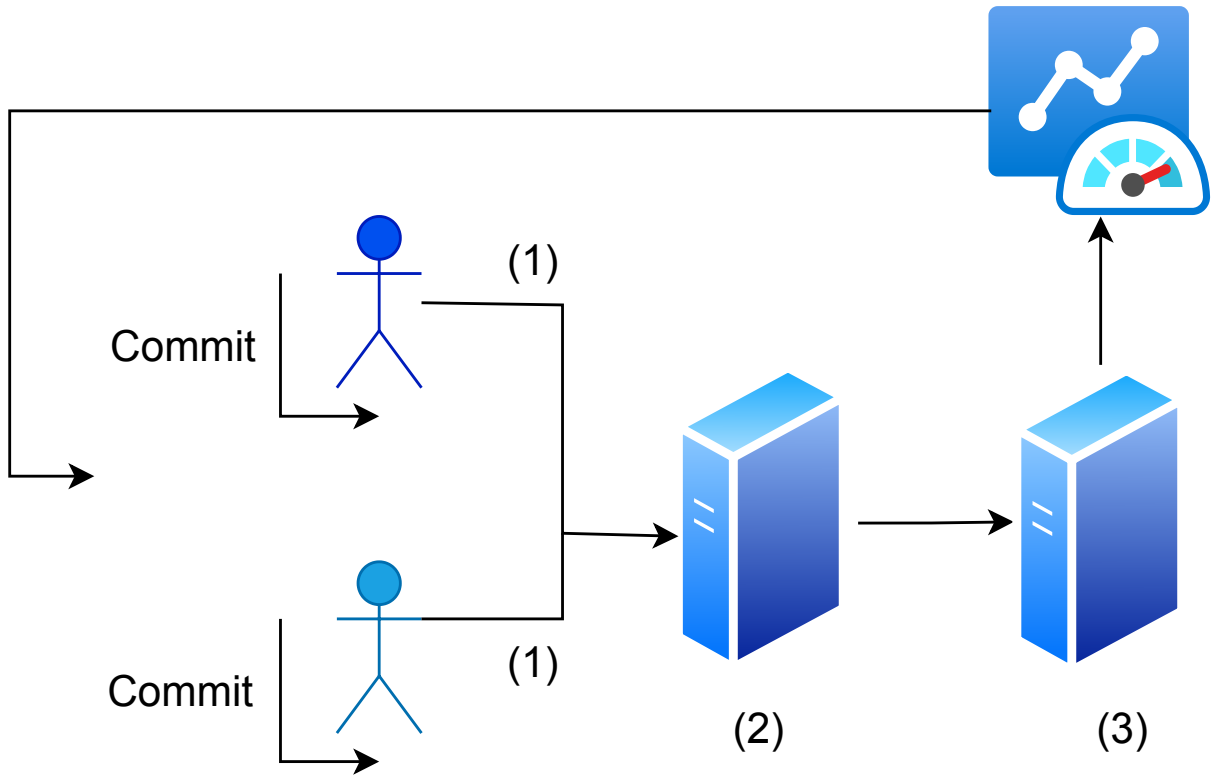


Figure 2: (a) What the Figure Represents, and What (1), (2), and (3) Could Denote for Making it Explicit?

A Few Short Mock-up Questions Again (5/7)

What is the Difference Between a Fault and a Failure?

A Few Short Mock-up Questions Again (6/7)

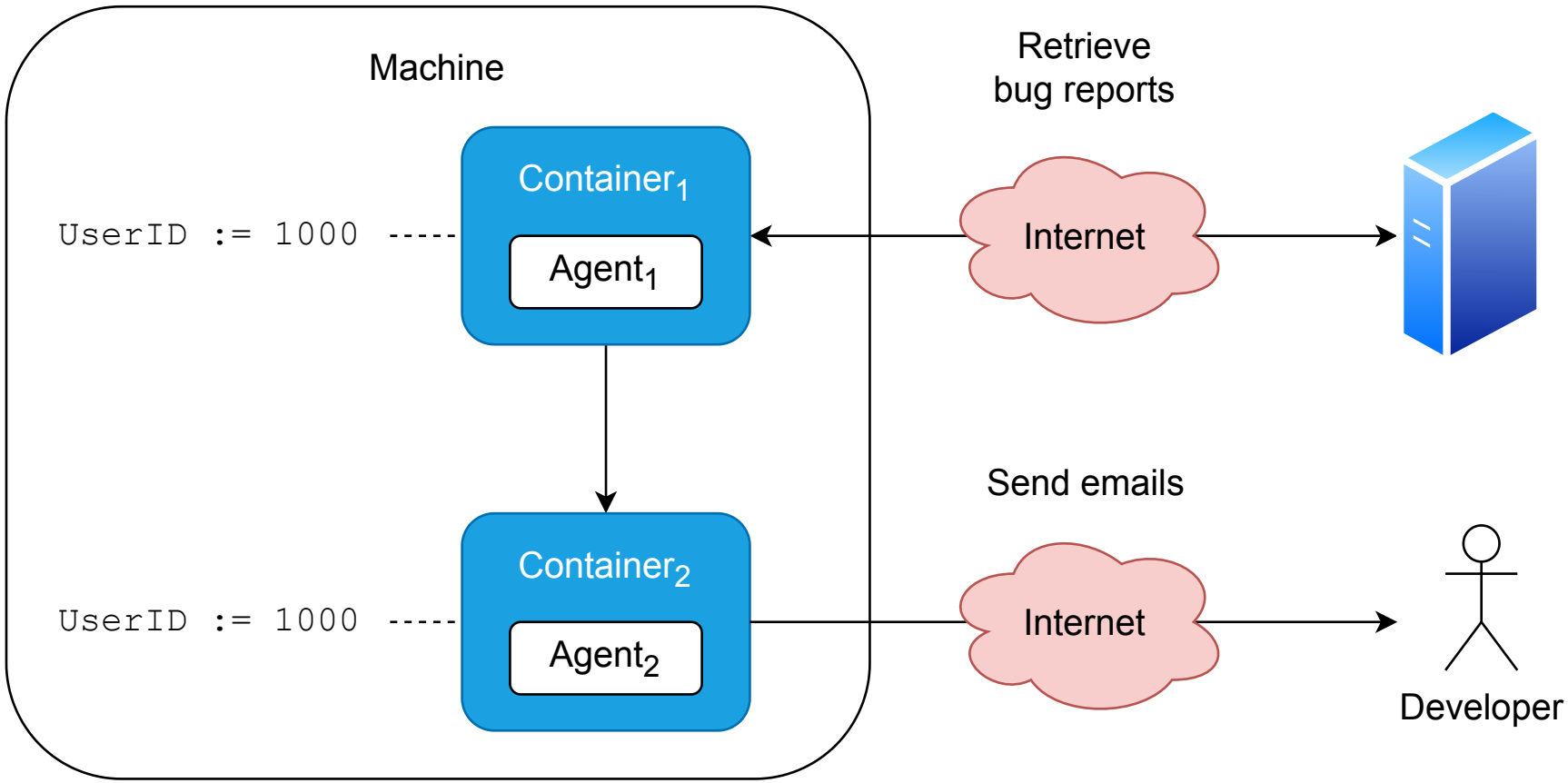


Figure 3: (a) What Design Tactic Is Already Done to Somewhat Secure the Design, (b) and What Might Be Done More? (Much so briefly.)

A Few Short Mock-up Questions Again (7/7)

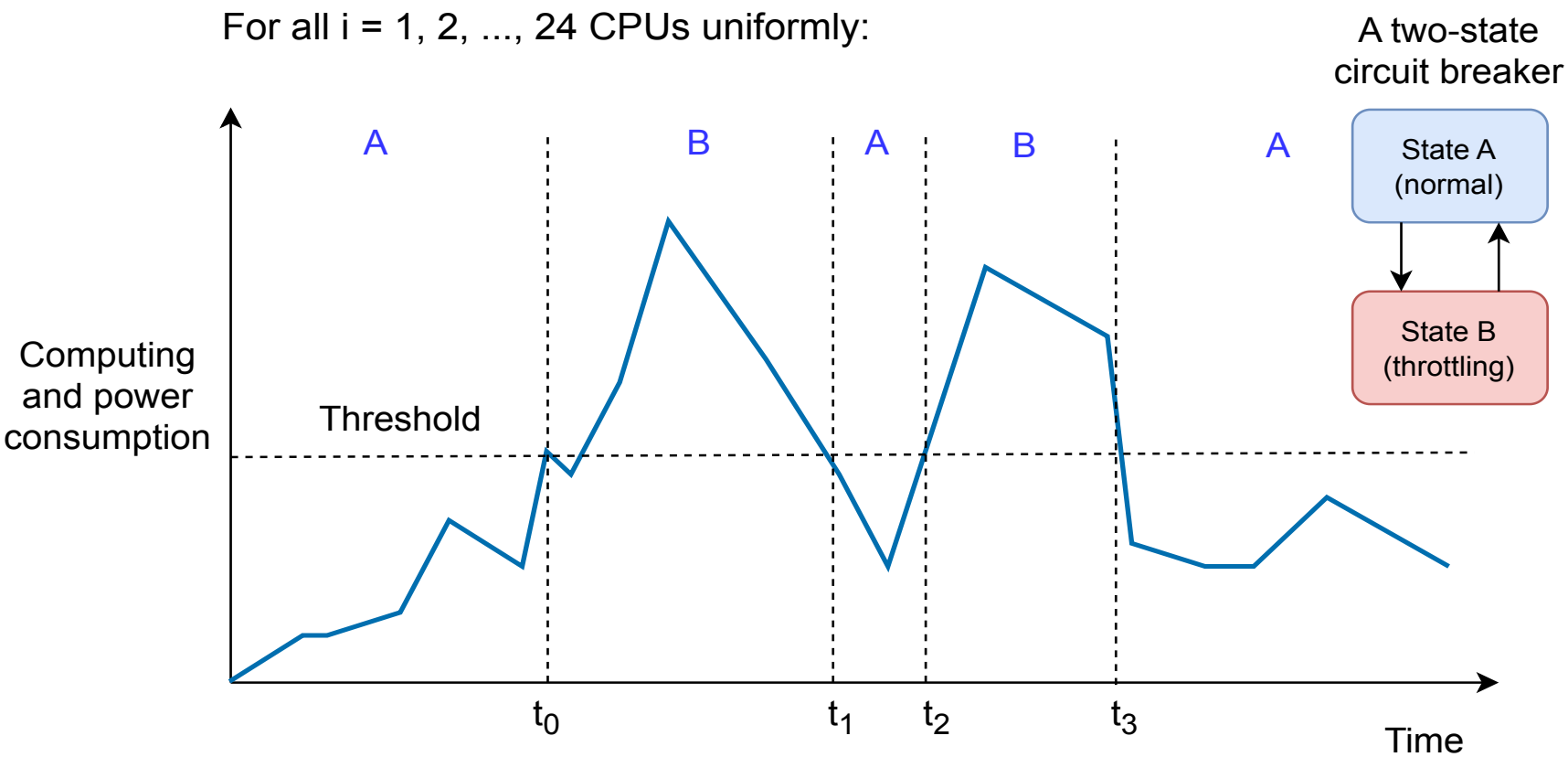


Figure 4: Could This Design Be Improved?

Today's Lecture

- Today's lecture is again about **cyber security**
- Specifically, we continue the **security engineering** theme but now without the agentic context

Question

What Kind of Trade-Offs Security Might Have?

Cyber Security Trade-Offs

Figure 5: Five Example Trade-Offs With Cyber Security

The Security Versus Reliability Trade-Off (1/2)

- Adkins et al. (2020) present the following: when "*designing a system to handle failure, you must balance between optimizing for reliability by failing open (safe) and optimizing for security by failing closed (secure)*"; i.e., you must solve:
  1. "*To maximize* **reliability**, *a system should* **resist failures and serve as much as possible**" in terms of access controls, firewalls, and alike; the philosophical default is "**allow all**"
  2. "*To maximize* **security**, *a system should* **lock down fully** *in the face of uncertainty*"; philosophically it is about "**deny all**"
- To solve the trade-off, they recommend that a **nonnegotiable security posture** must be greed upon in an organization

The Security Versus Reliability Trade-Off (2/2)

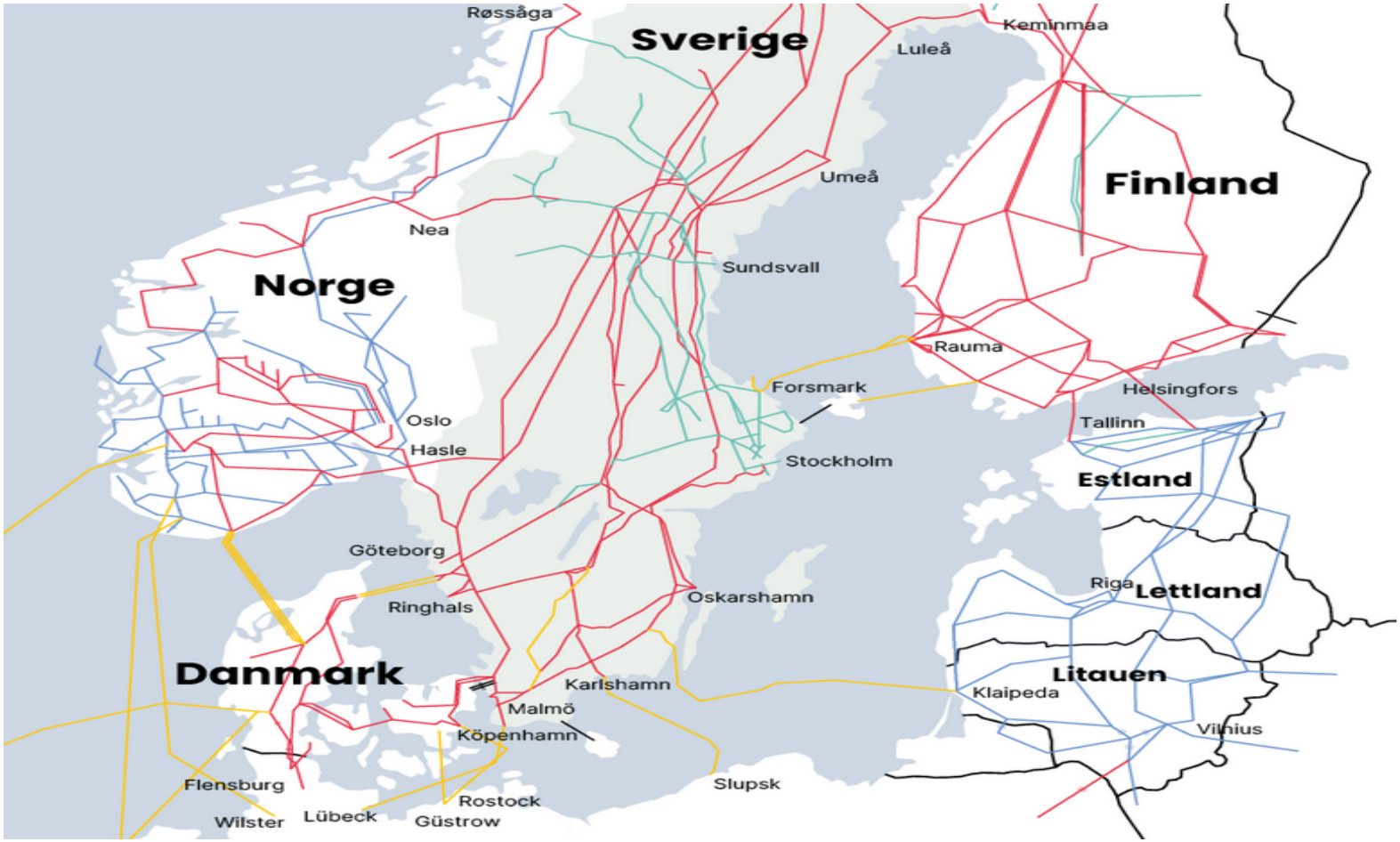


Figure 6: https://www.svk.se/om-kraftsystemet/om-transmissionsnatet/transmissionsnatskarta/

- Would You Maximize Reliability or Security?

Cyber Security as a Constraint (1/4)

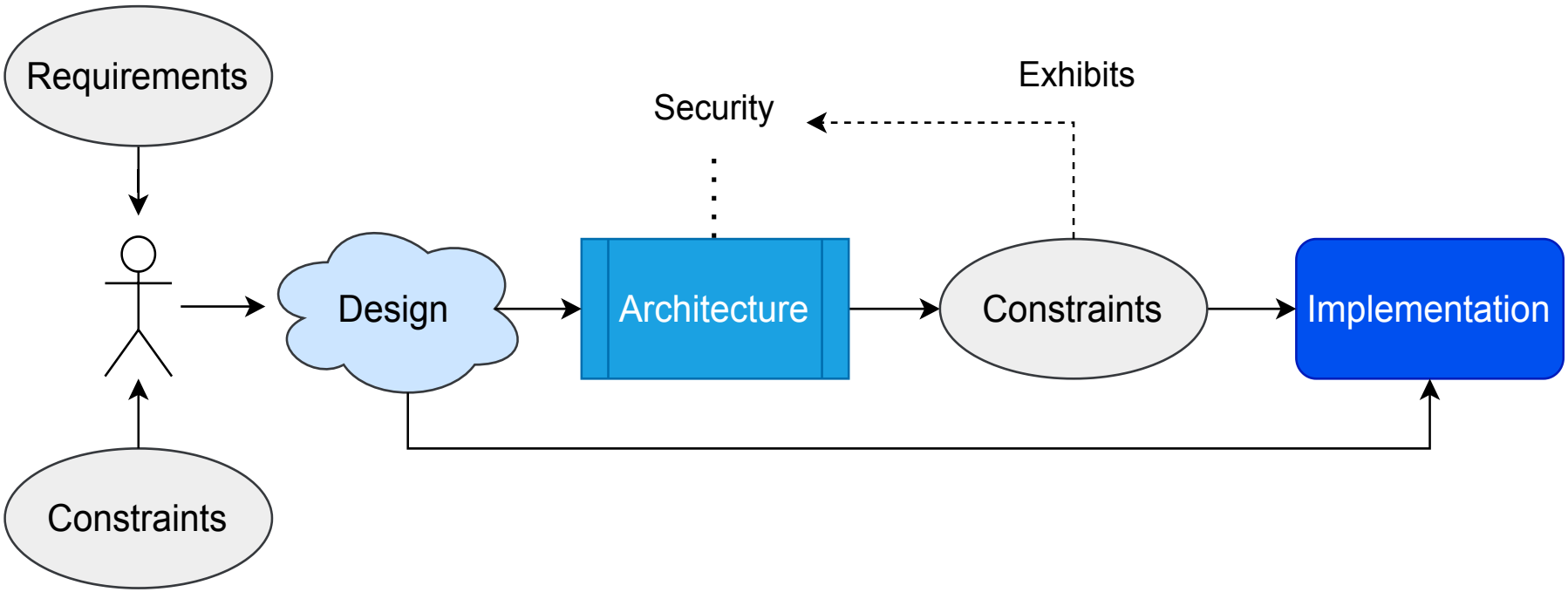


Figure 7: What Might Be an Example Constraint Here?

Cyber Security as a Constraint (2/4)

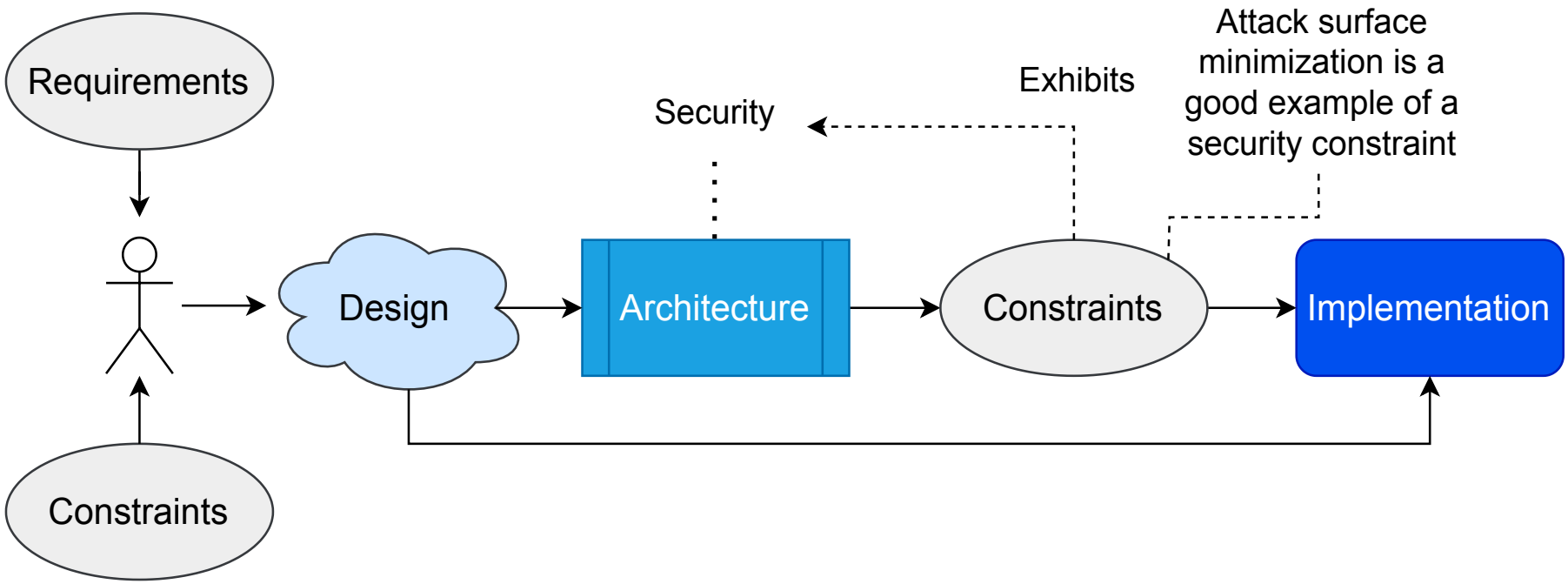


Figure 8: Should Something Else Be Constrained Too?

Cyber Security as a Constraint (3/4)

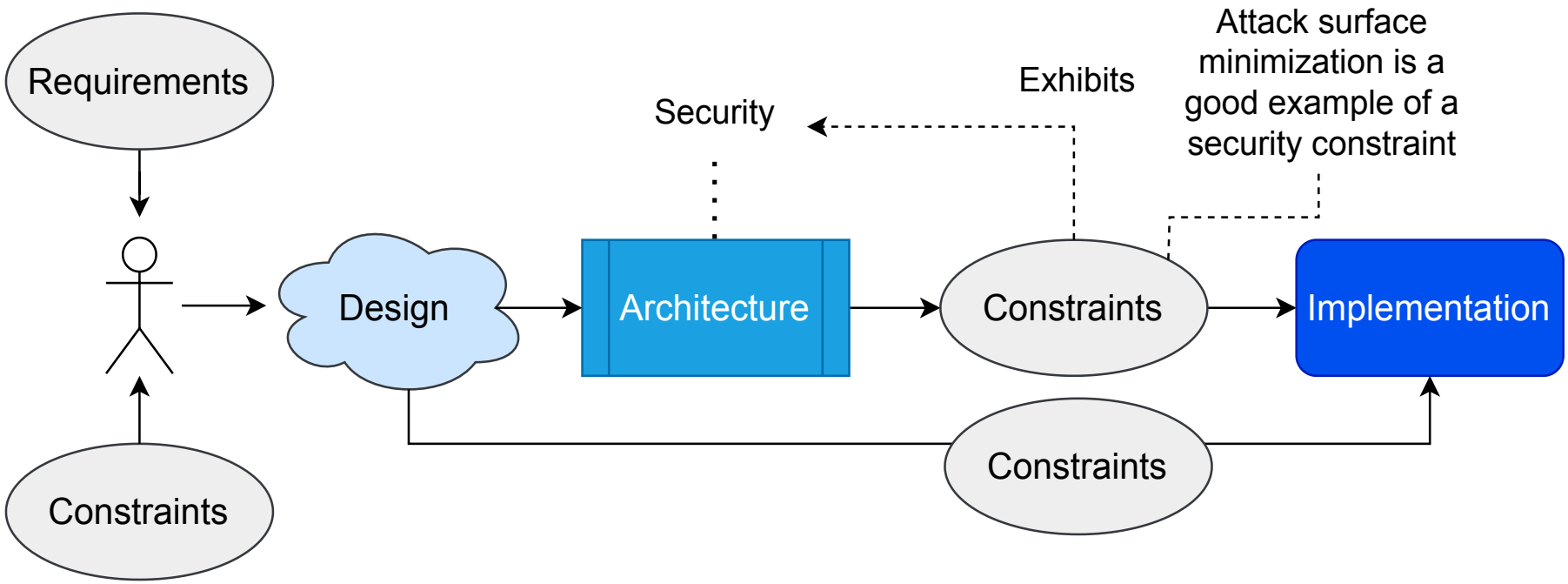


Figure 9: What Might Be an Example Constraint Here?

Cyber Security as a Constraint (4/4)

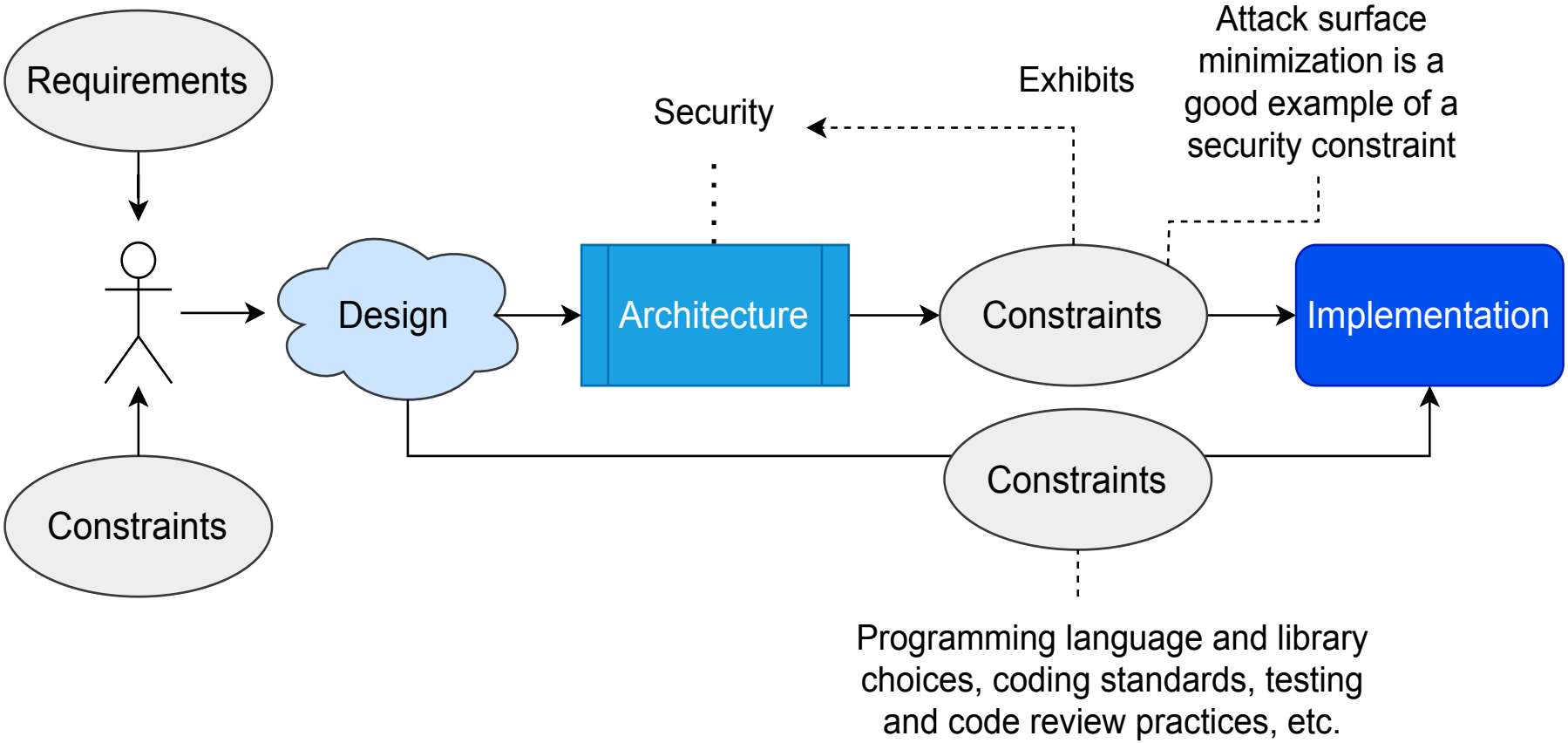


Figure 10: Security Should Constraint also Detailed Design

Microsoft's SDL Model (1/9)

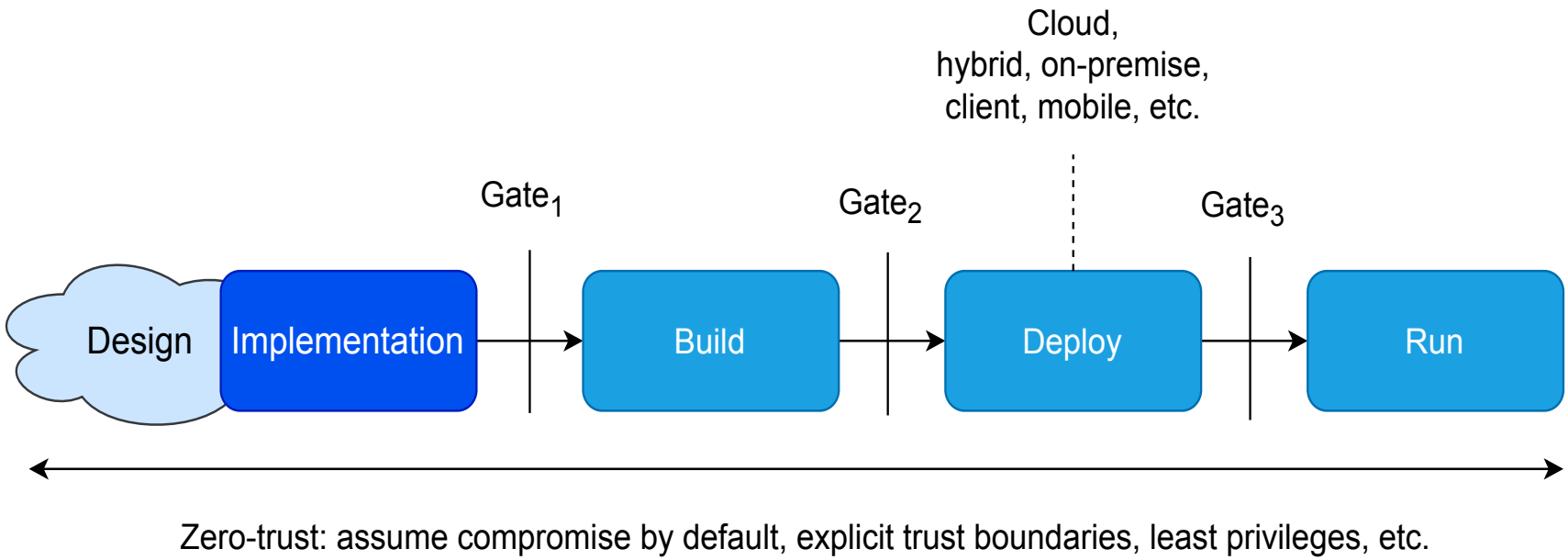


Figure 11: Microsoft's (2026a) Current Version of its Security Development Lifecycle (SDL) Model

Microsoft's SDL Model (2/9)

- Regarding the three gates in the previous figure, Gate_1 is there to provide checks and balances on development-time security practices (e.g., whether vulnerabilities have been introduced)
  - Note that **developers' operational security** is important!
- Gate_2 is there to ensure that no security issues arise from CI/CD pipelines and related development infrastructures
- Gate_3 is there to emphasize the role of **secure configurations**, **hardening**, and related practices

Microsoft's SDL Model (3/9)

- The warning from Zimmermann (2019) is still valid and increasingly relevant regarding operational security of developers; **social engineering** is used for both package takeovers (e.g., seeking a maintainer role) and account takeovers (e.g., keys, repository credentials, etc.)

Figure 12:

https://opensourcemalware.com/blog/social-engineering-playbook

Microsoft's SDL Model (4/9)

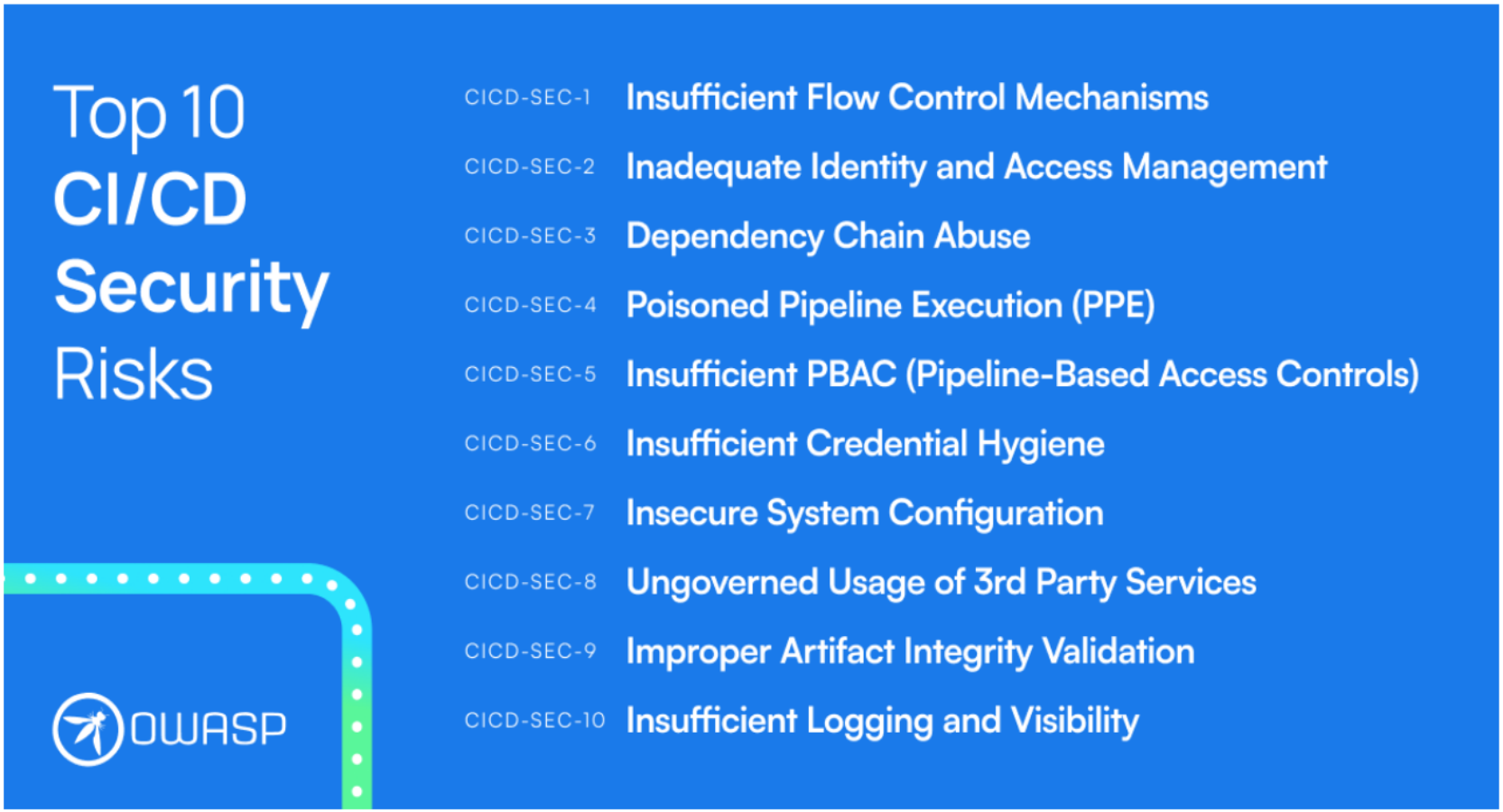


Figure 13: https://owasp.org/www-project-top-10-ci-cd-security-risks/

Microsoft's SDL Model (5/9)

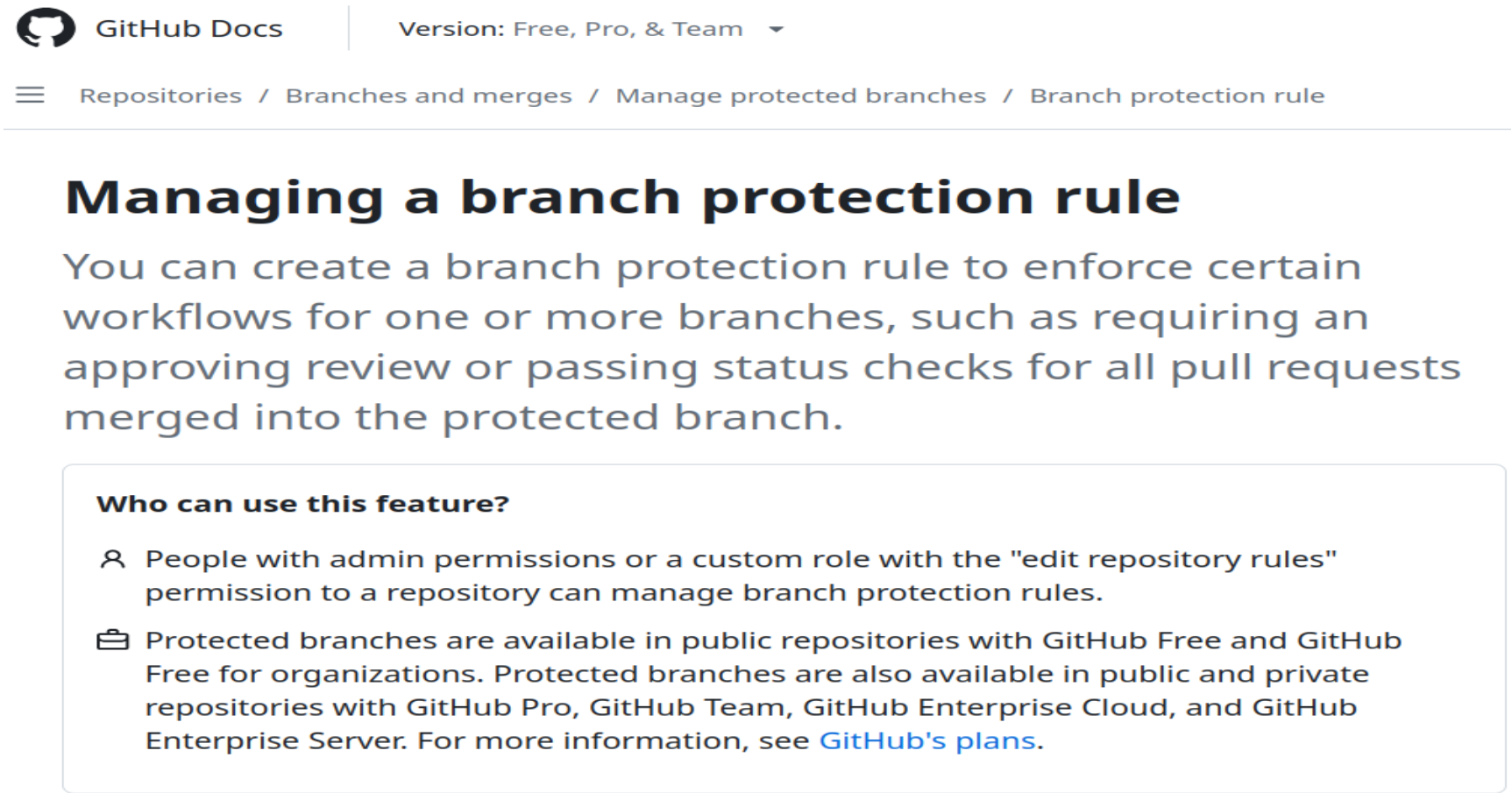


Figure 14: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/

Microsoft's SDL Model (6/9)

- In addition, OWASP recommends limiting **auto-merge rules**
- Furthermore and particularly importantly, they recommend **prior approval of a CI/CD account** for artifacts to flow through the whole CI/CD pipeline
  - In particular, *prevent non-authorized accounts for triggering a production build and subsequent deployment*
  - When dealing with production branches, **prior approvals and reviews** are in other words a good idea

Microsoft's SDL Model (7/9)

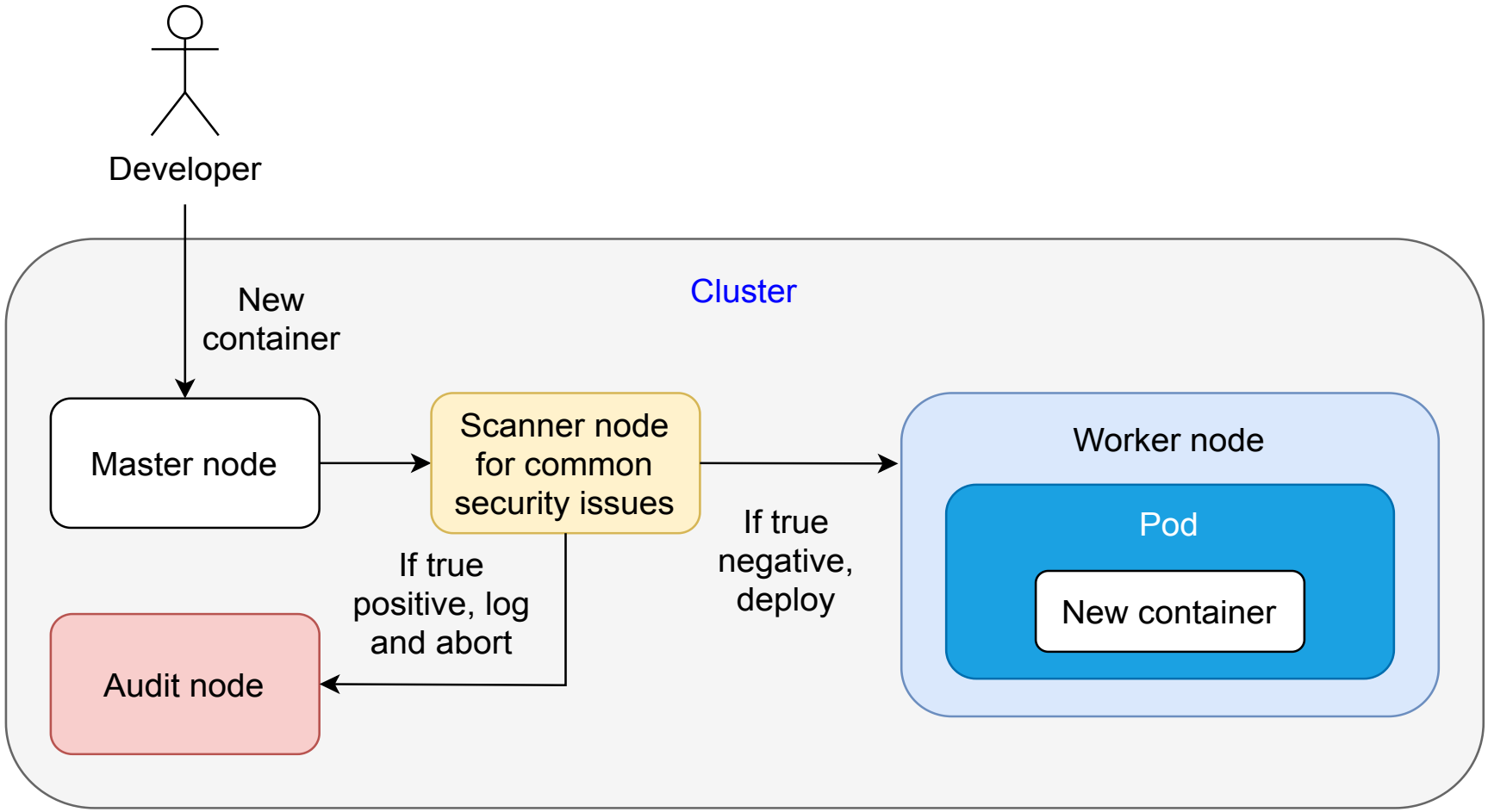


Figure 15: One Option (Among Many) for Scanning Kubernetes Cluster

Microsoft's SDL Model (8/9)

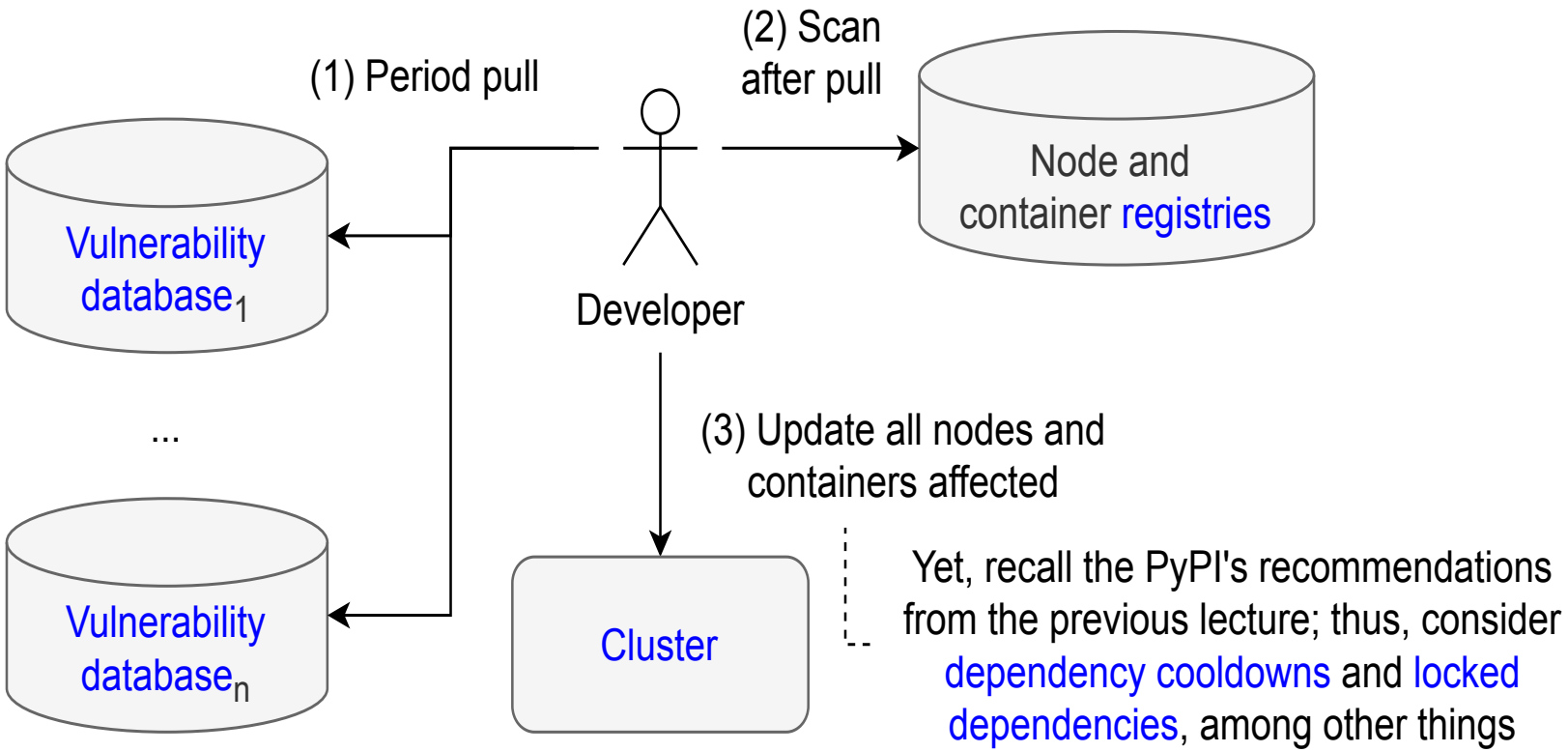


Figure 16: Another Option for Scanning Kubernetes Cluster

Microsoft's SDL Model (9/9)

- It is a good idea to run also **hardening tools** to check potential **configuration vulnerabilities** of a deployment
  - If the deployment's configuration changes, these could be triggered automatically too

- These tools vary from a deployment context to another; for Kubernetes alone, there are probably more than ten such tools
  - For instance, Cheukov, Kubeaudit, KubeLinter, Kube-score, Kubesec, SLI-KUBE, Kube-bench, Kubescape, Trivy, Neuvector, StackRox, and probably many more

Risks, Again (1/2)

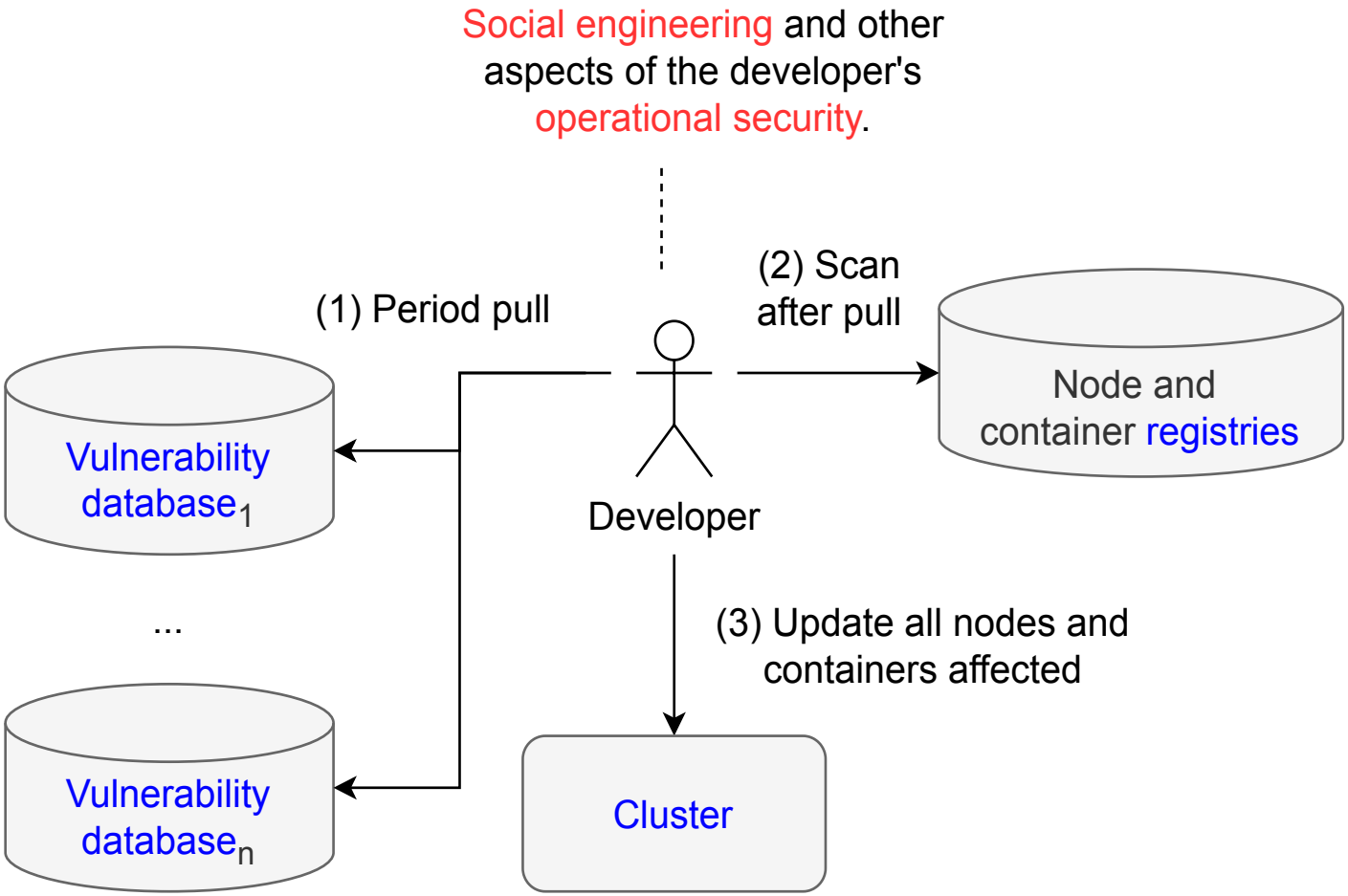
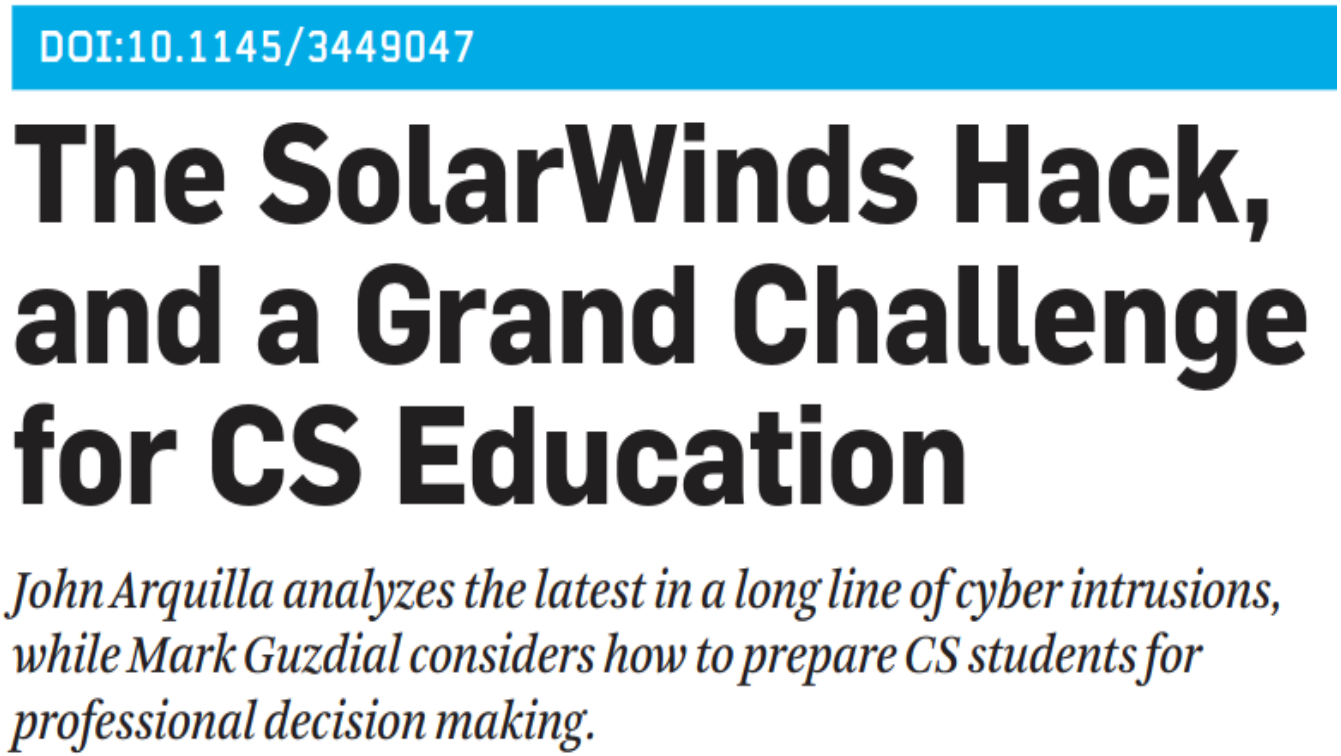


Figure 17: What Additional Risks There Might Be and Where?

Risks, Again (2/2)



DOI:10.1145/3449047

The SolarWinds Hack, and a Grand Challenge for CS Education

John Arquilla analyzes the latest in a long line of cyber intrusions, while Mark Guzdial considers how to prepare CS students for professional decision making.

Figure 18: https://dl.acm.org/doi/pdf/10.1145/3449047

Security During Design Phase (1/3)

- Microsoft (2026a) recommends four tasks:
  1. Identify relevant **security standards**
  2. Identify and define **security requirements**
  3. Define **metrics for compliance reporting**
  4. Create a process for **security exception process**
- Of these, (2) involves also defining constraints for detailed design and implementation (cf. top-vulnerability rankings)
- Of these, (4) means that you should have a process for documenting and approving any and all deviations from security standards and requirements

  • To do that, you must be able to determine what is the **acceptable level of risk** caused by a given deviation

Security During Design Phase (2/3)

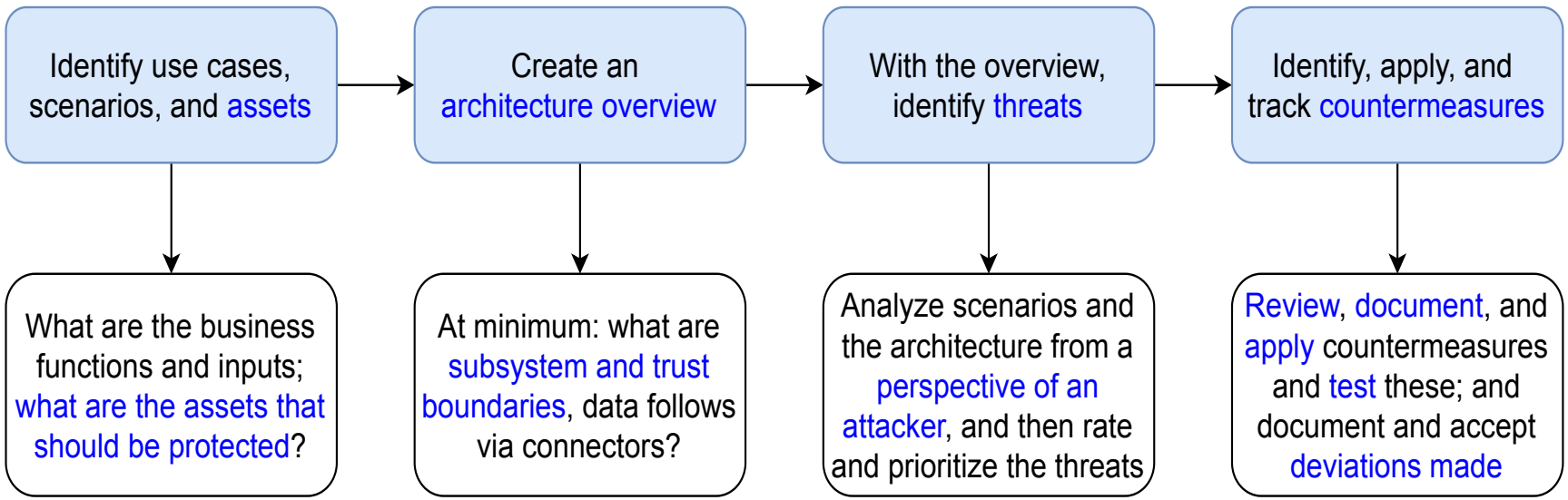


Figure 19: Threat Modeling Tasks (adopted from Microsoft 2026)

Security During Design Phase (3/3)

Authors MUST describe

1. which attacks are out of scope (and why!)
2. which attacks are in-scope
2.1 and the protocol is susceptible to
2.2 and the protocol protects against

Figure 20: https://www.rfc-editor.org/rfc/rfc3552

Trust Boundaries Revisited (1/3)

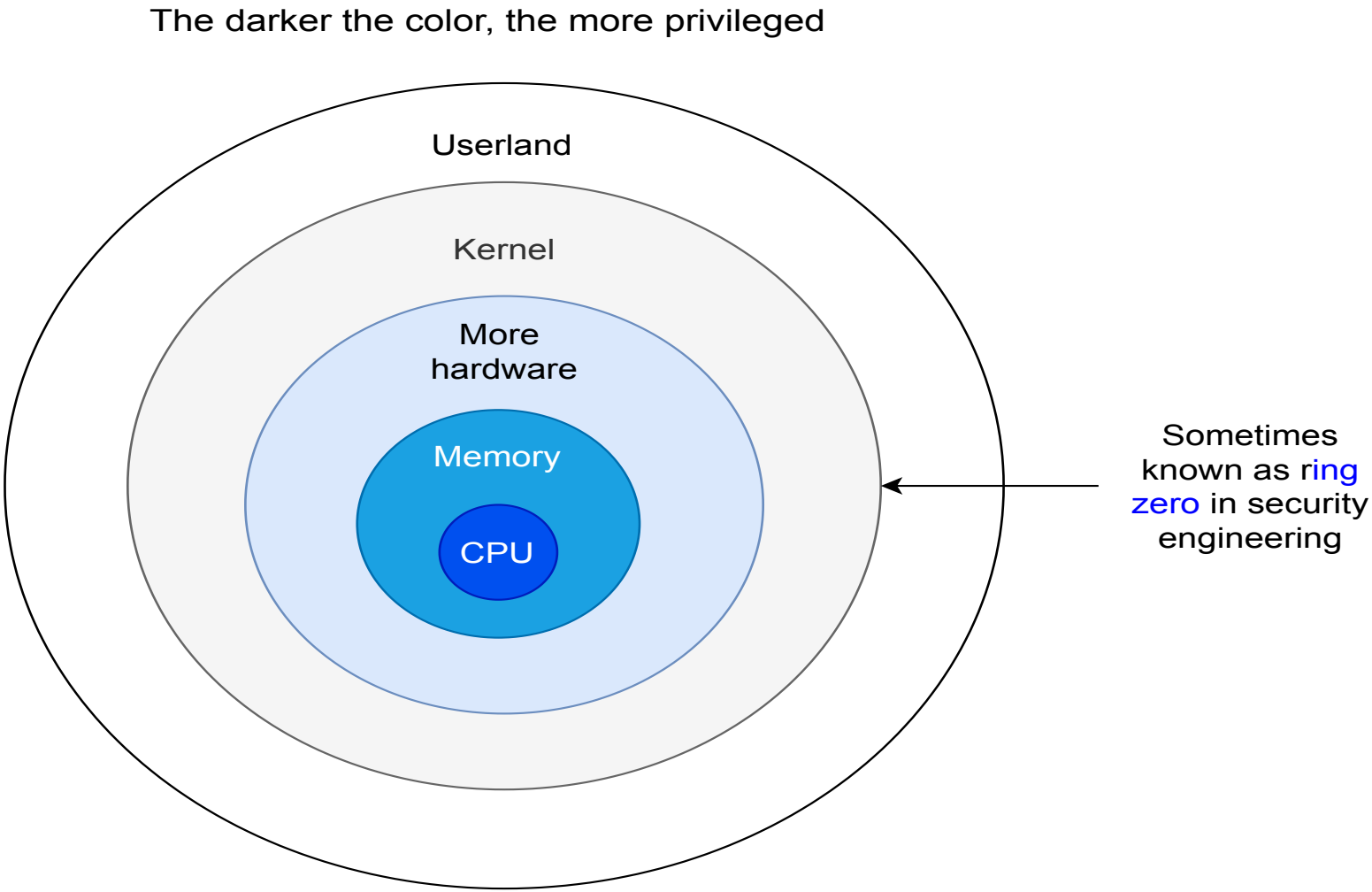


Figure 21: Low-Level Trust Boundaries (Protection Rings)

Trust Boundaries Revisited (2/3)

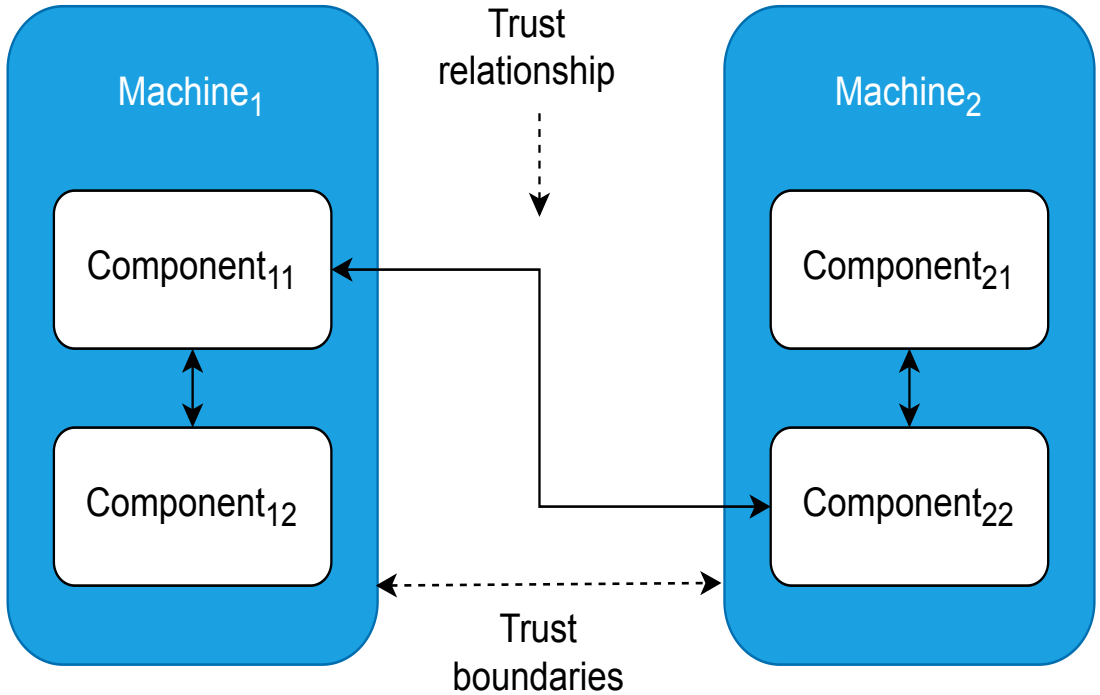


Figure 22: A Simple Example of Trust Boundaries and Trust Relationships

Trust Boundaries Revisited (3/3)

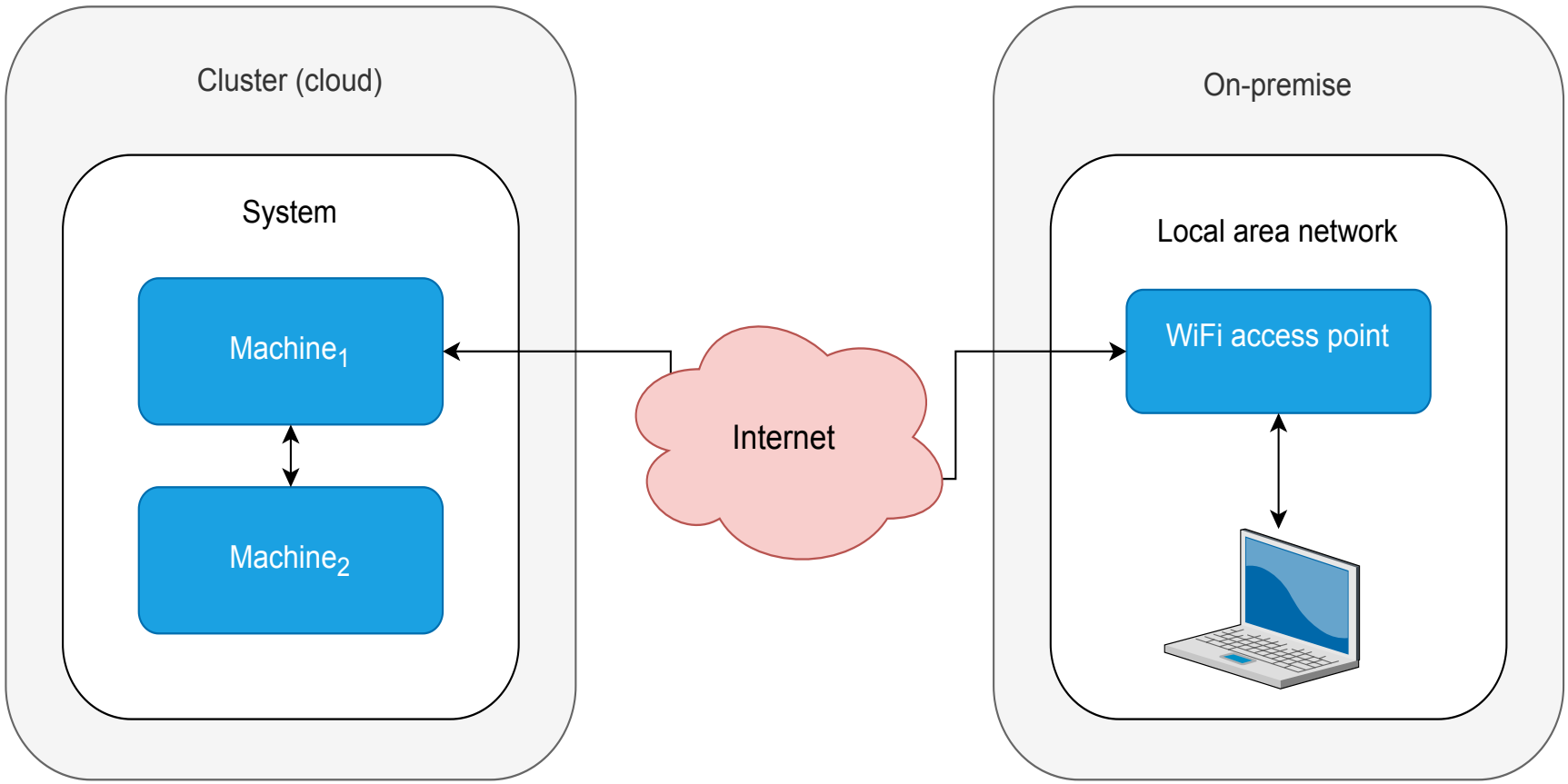


Figure 23: Where Are the Trust Boundaries?

The Kill Chain: A Worked Example (1/8)

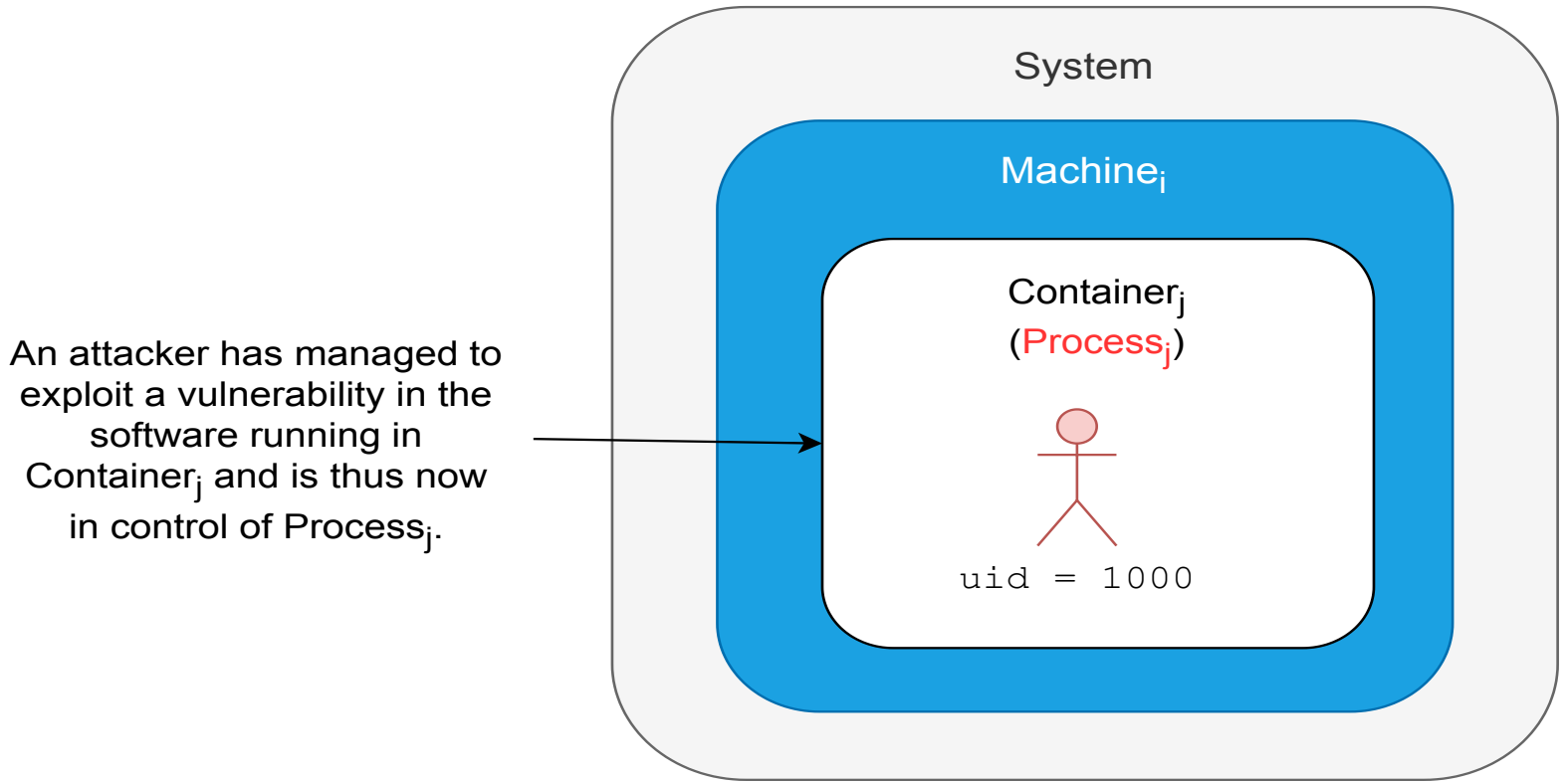


Figure 24: How the Attack Would Perhaps Proceed?

The Kill Chain: A Worked Example (2/8)

The attacker has managed to further conduct a privilege escalation attack by exploiting another vulnerability and is now running with root privileges and in control of Process_j.

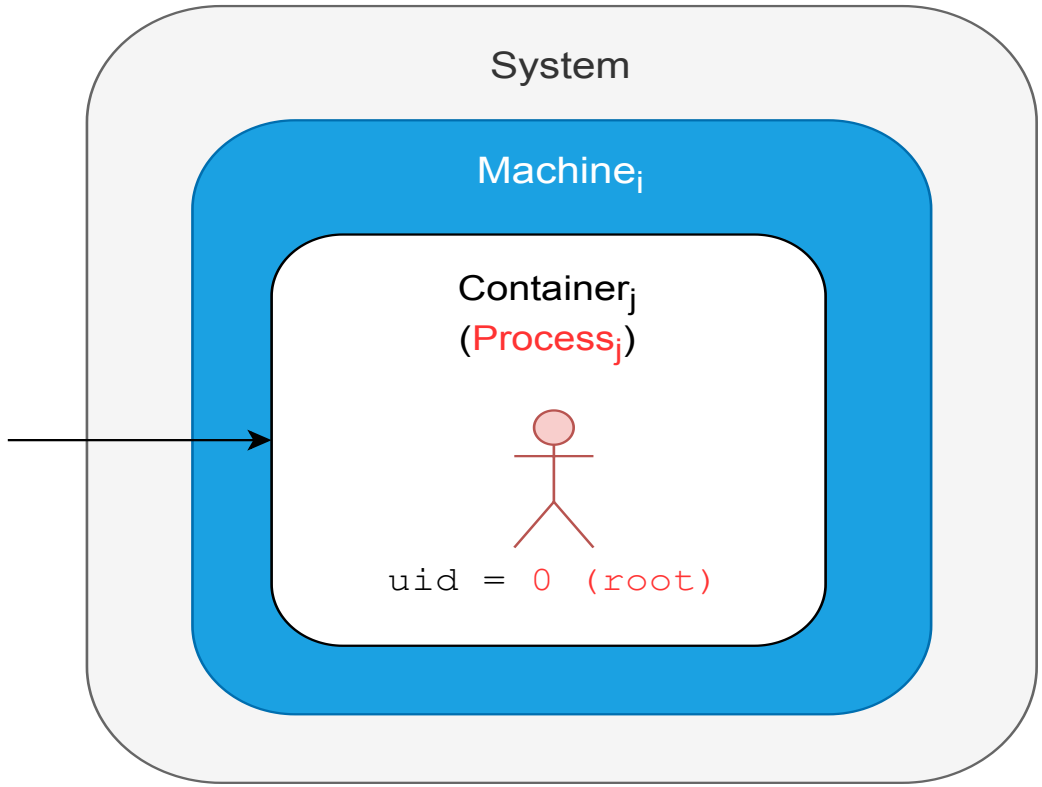


Figure 25: How the Attack Would Now Likely Proceed?

The Kill Chain: A Worked Example (3/8)

The attacker has managed to exploit yet another vulnerability now in the container system and is thus now in control of Process_j that is no longer in any container. As the attacker has root privileges, she is effectively in control of the whole Machine_i too.

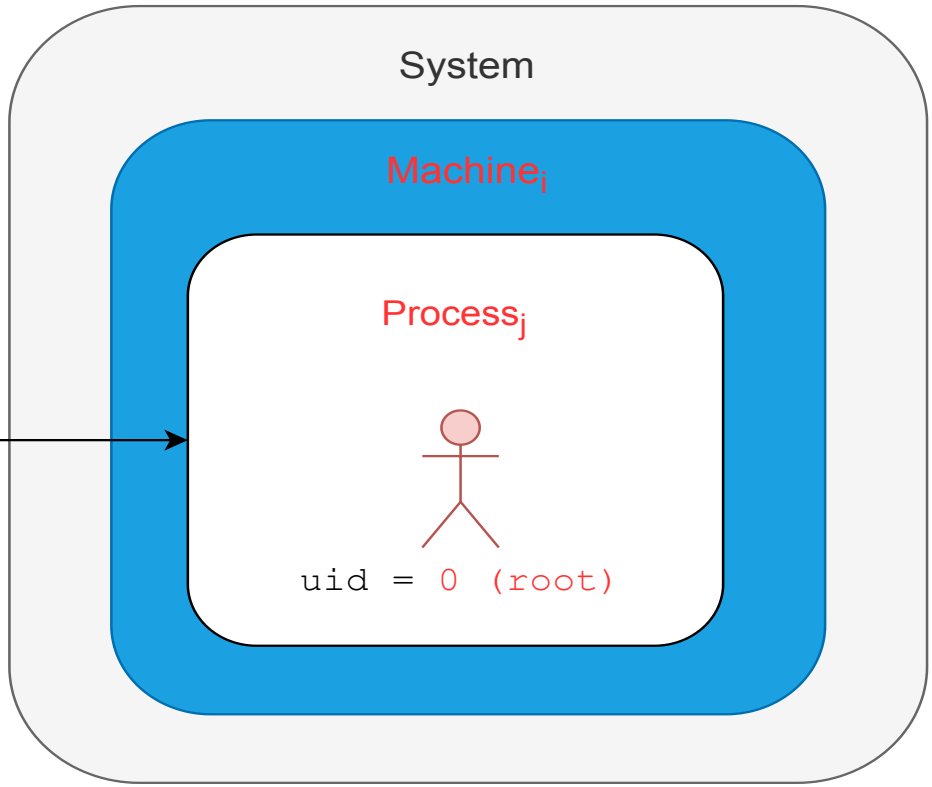


Figure 26: How the Attack Would Now Perhaps Proceed?

The Kill Chain: A Worked Example (4/8)

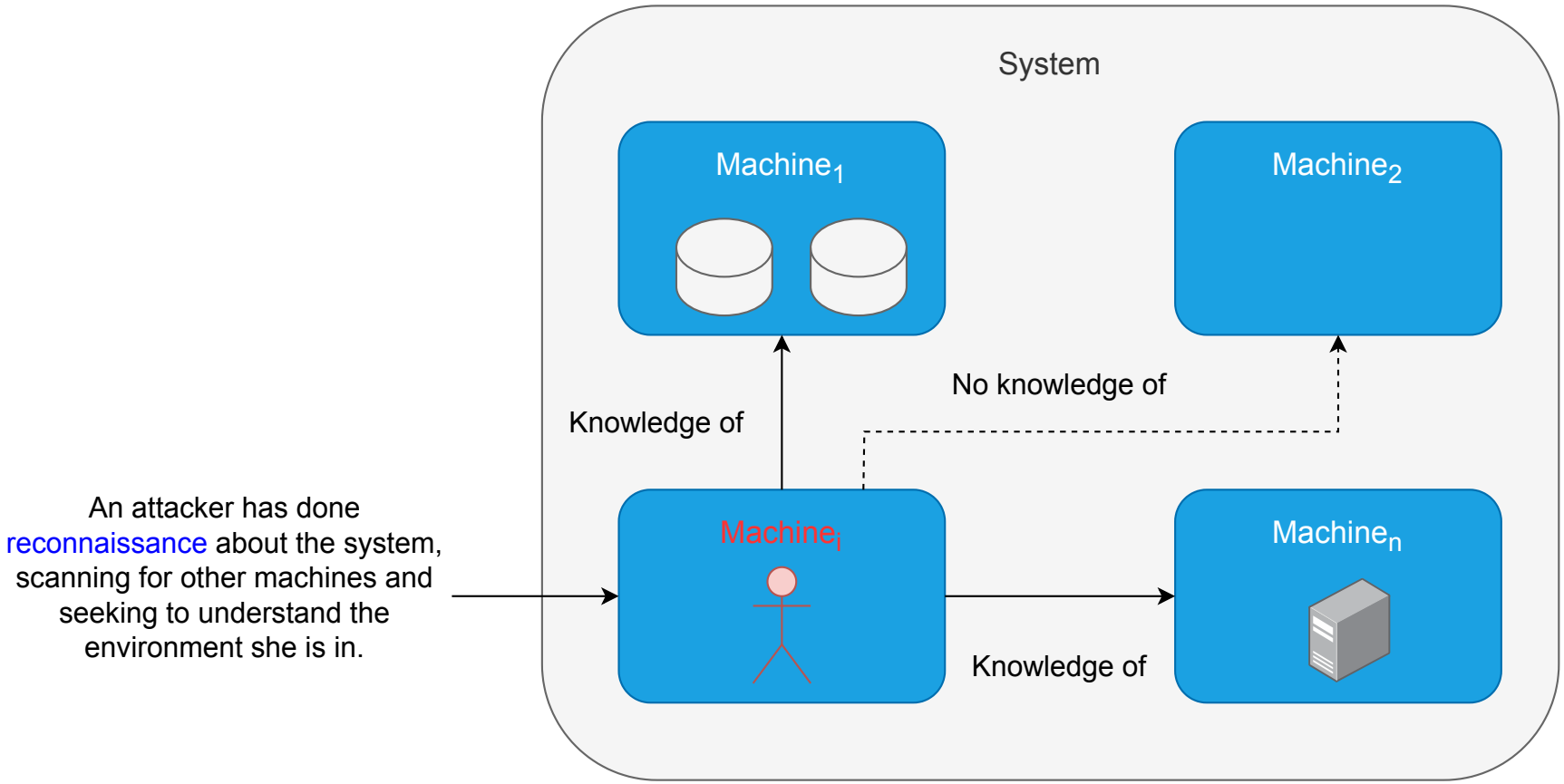


Figure 27: How the Attack Might Now Proceed?

The Kill Chain: A Worked Example (5/8)

The attacker exploited the same vulnerability chain as previously and gained control of Machine_1 too.

After having control, she established a strong encrypted tunnel via a compromised proxy machine to her malicious system in the dark web. Then she transferred all data in Machine_1 to Machine_k, and closed the tunnel.

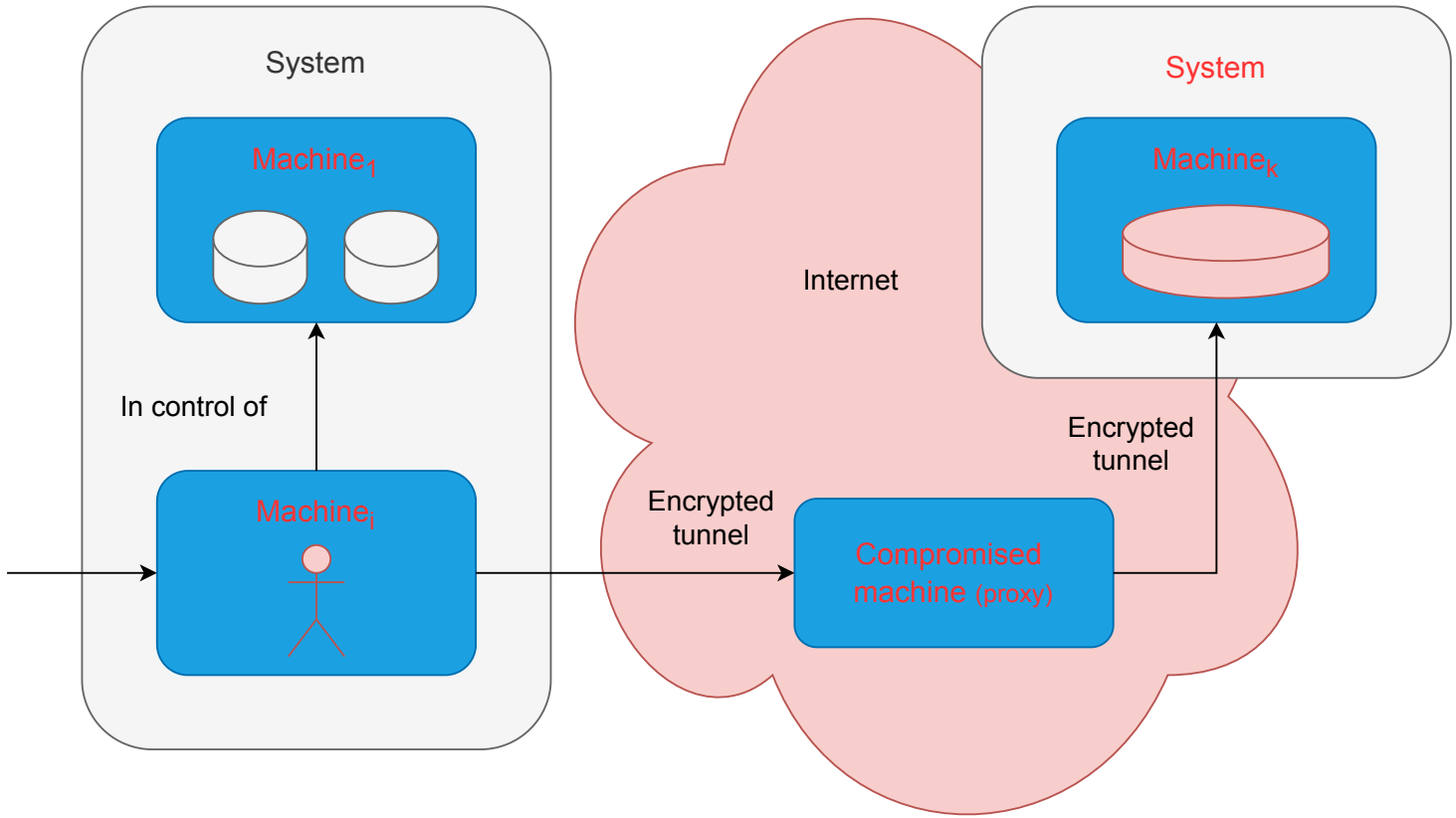


Figure 28: How the Attack Might Now Proceed?

The Kill Chain: A Worked Example (6/8)

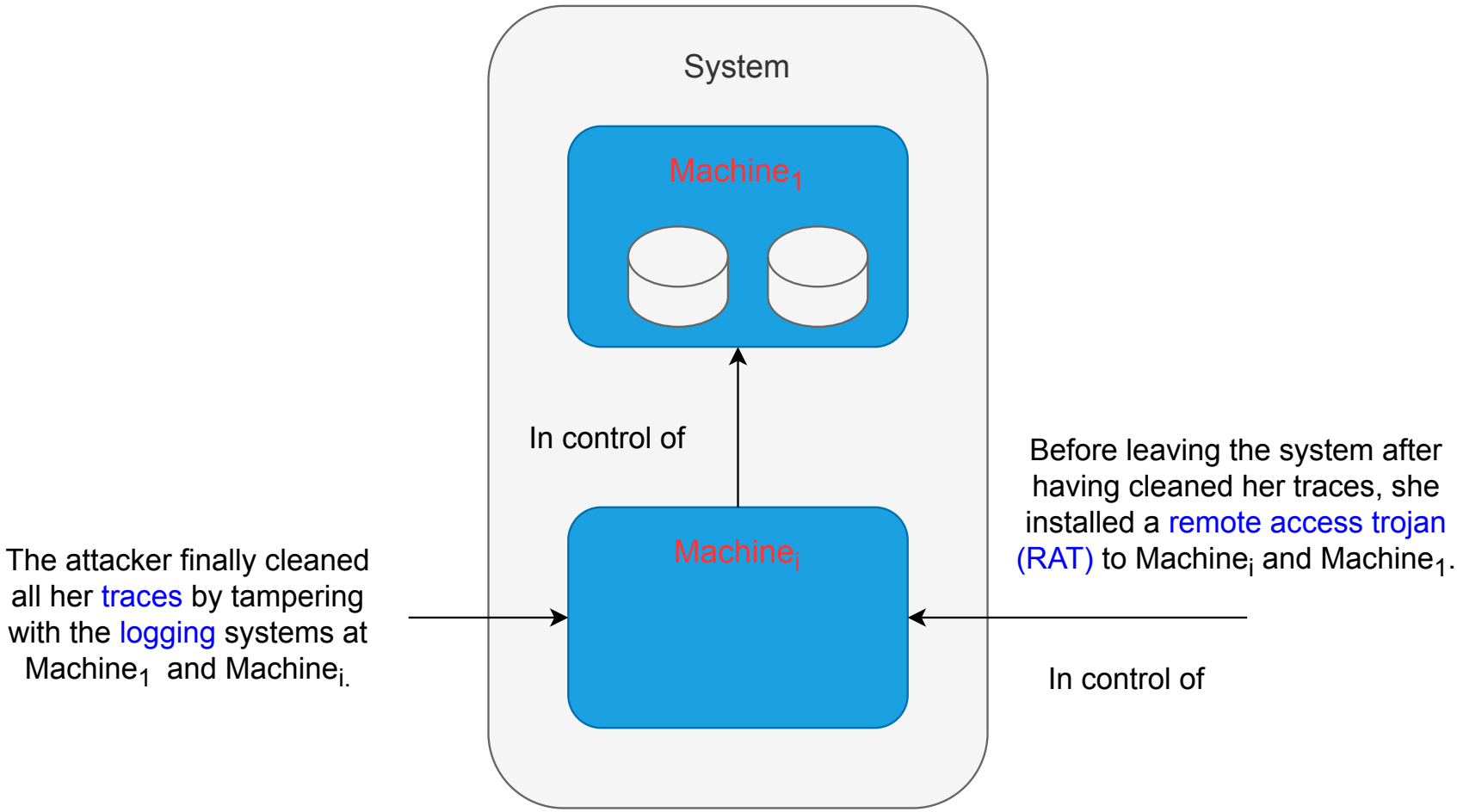


Figure 29: Ending the Chain

The Kill Chain: A Worked Example (7/8)

- The previous worked example illustrates various important points, among which are:
  1. **Exploitation chains** are often needed for a full compromise
  2. **Privilege escalation** is often needed for a full compromise
  3. It is difficult but possible to **break isolation** mechanisms
  4. **Reconnaissance** is needed for attackers to **move laterally**
  5. It is important to **design logging well**
  6. The **zero trust model** is important theoretically because it also demonstrates that simple and traditional "**demilitarized zones**" (DMZs) are not enough

The Kill Chain: A Worked Example (8/8)

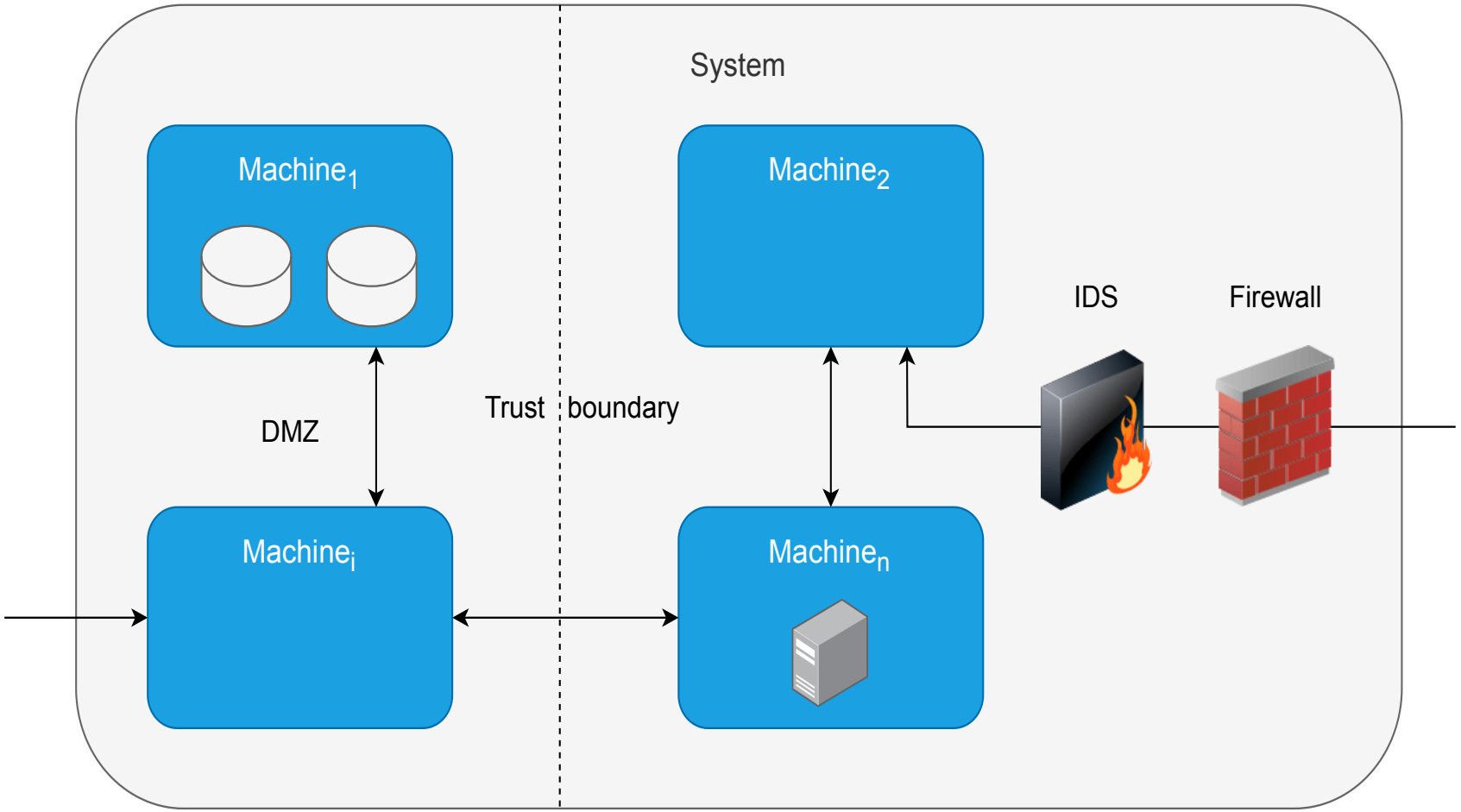


Figure 30: So, Perhaps the System (Deployment) Design Was Poor Too

Validation Revisited (1/3)

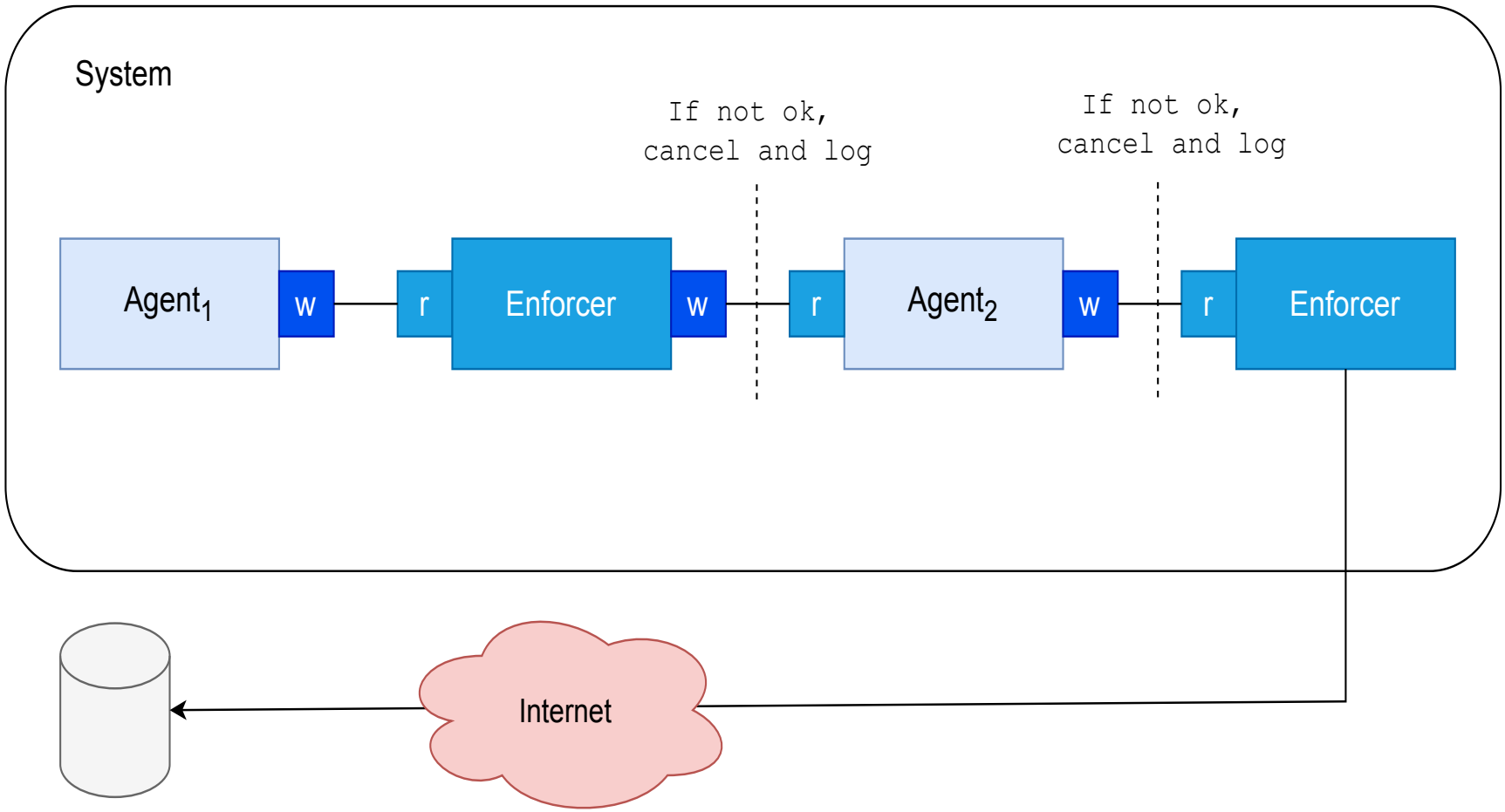


Figure 31: How It Might Be Improved?

Validation Revisited (2/3)

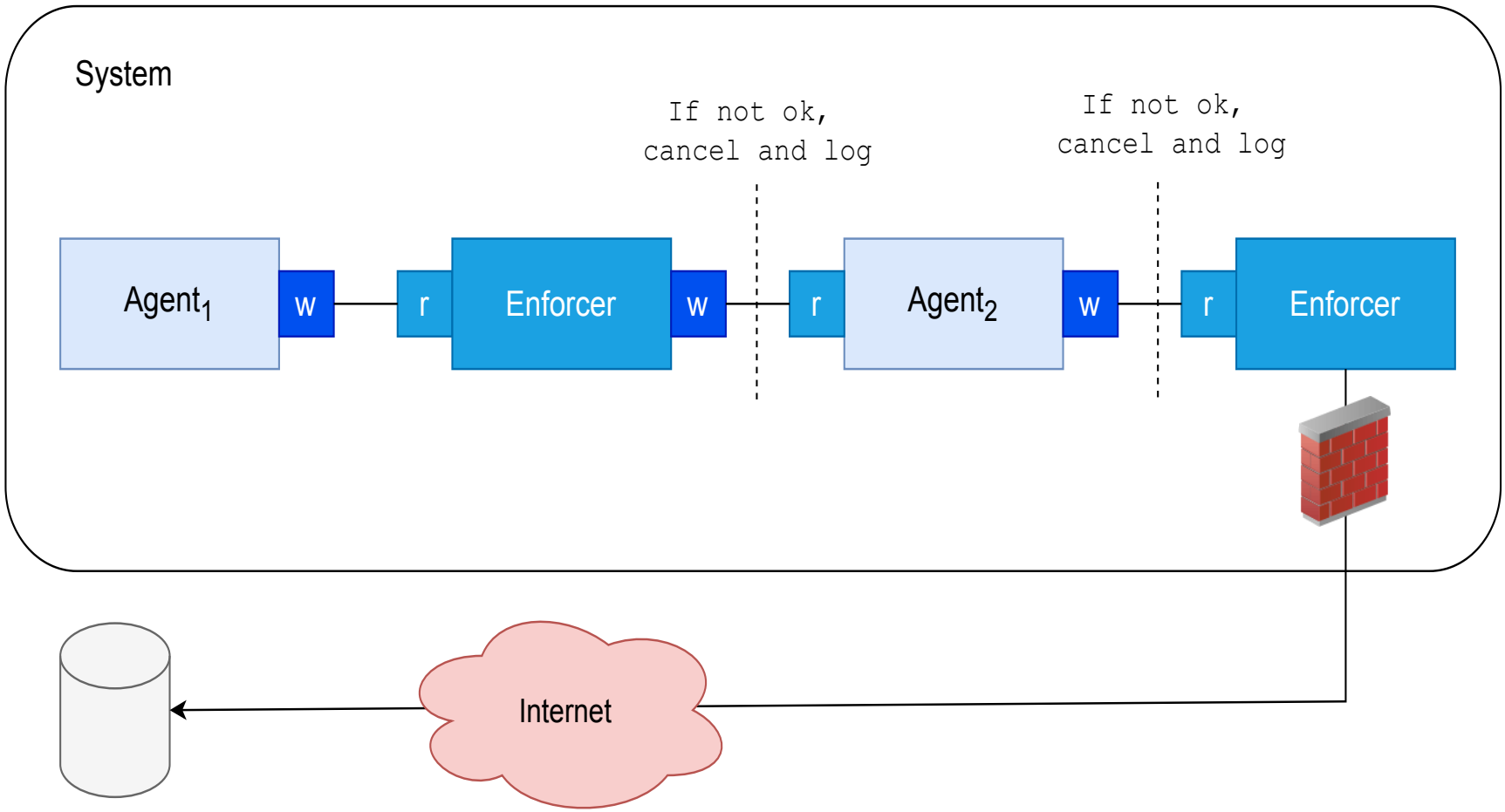


Figure 32: How It Might Be Still Improved?

Validation Revisited (3/3)

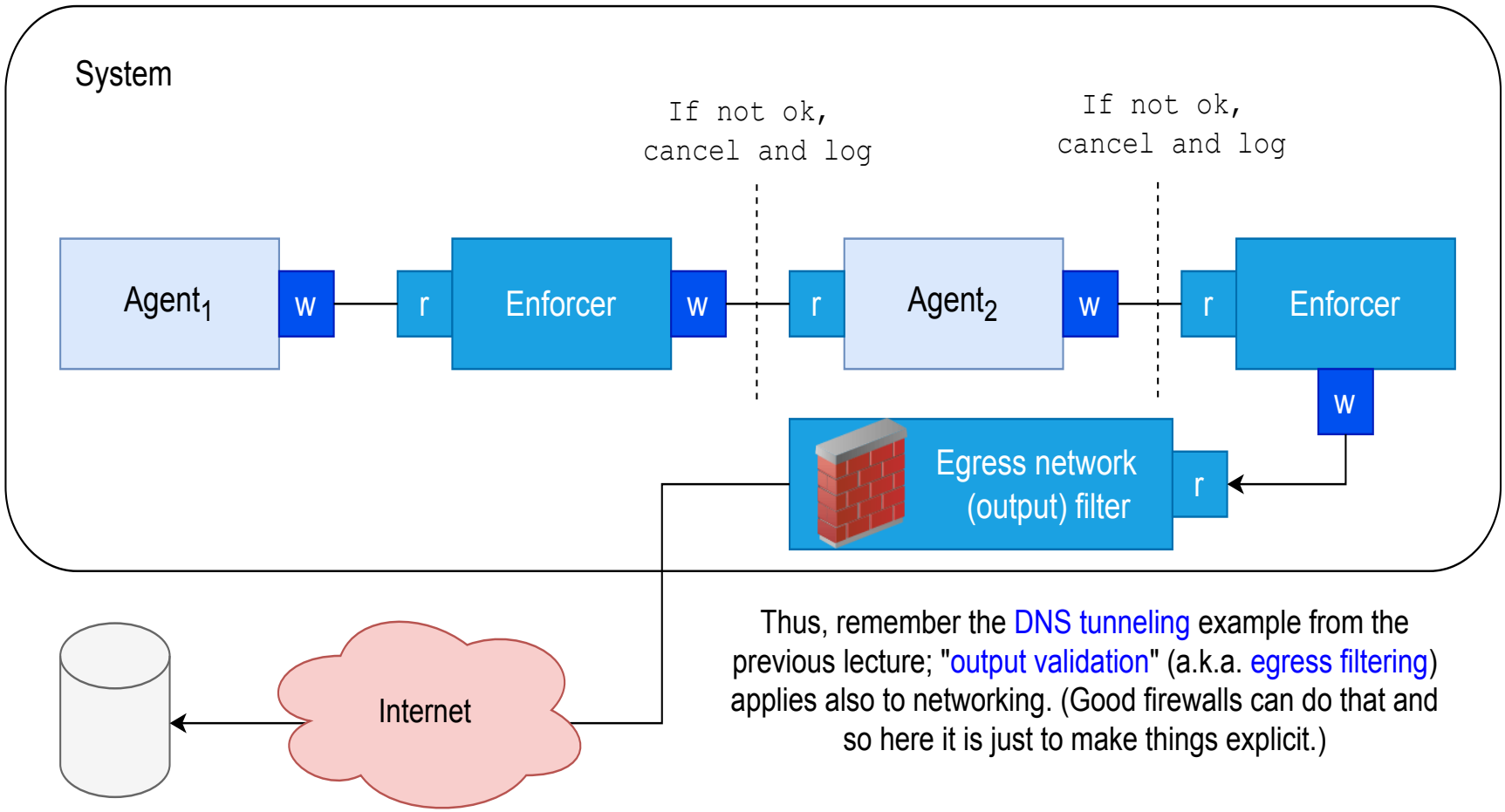


Figure 33: Thus, Recall the Zero Trust Principle w.r.t. Agent_1 and Agent_2

Layering Works Also for the Zero-Trust Principle

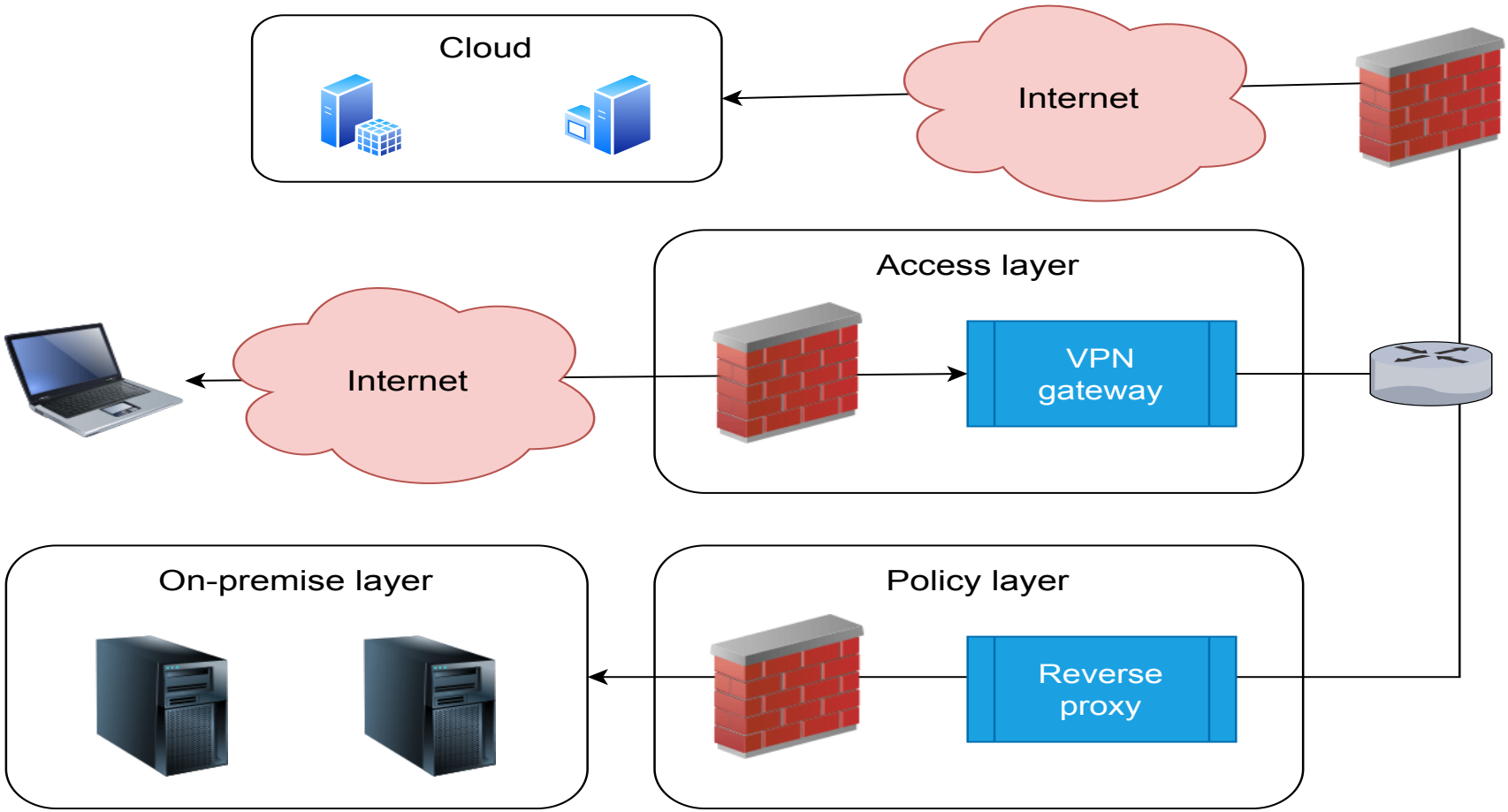


Figure 34: A Simple Zero-Trust Architecture (adopted from NCSC 2025)

Sidecars and the Zero-Trust Principle (1/5)

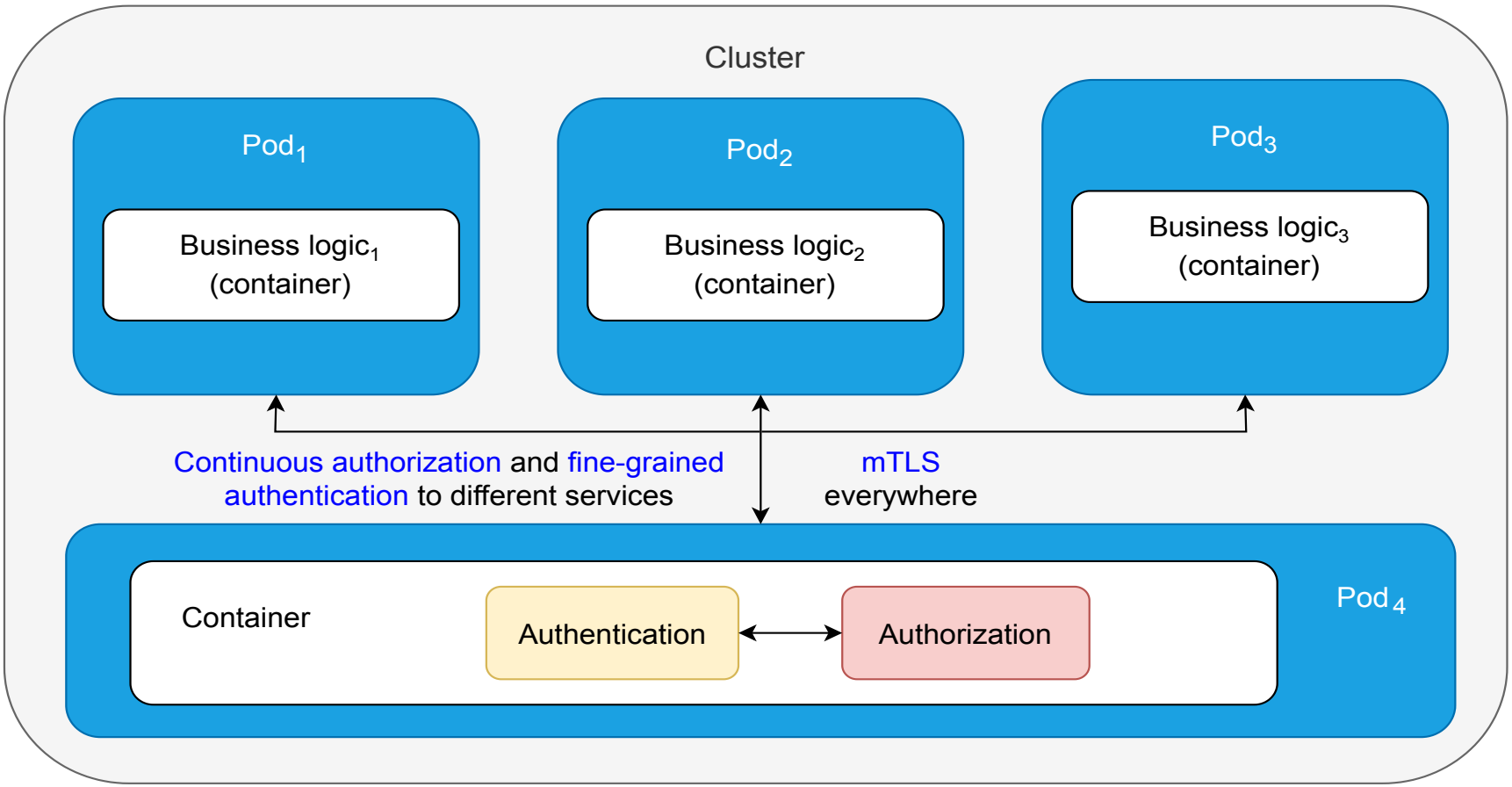


Figure 35: As the Zero-Trust Principle Promotes Continuous Authentication and Authorization, the Sidecar Pattern May Work Well

Sidecars and the Zero-Trust Principle (2/5)

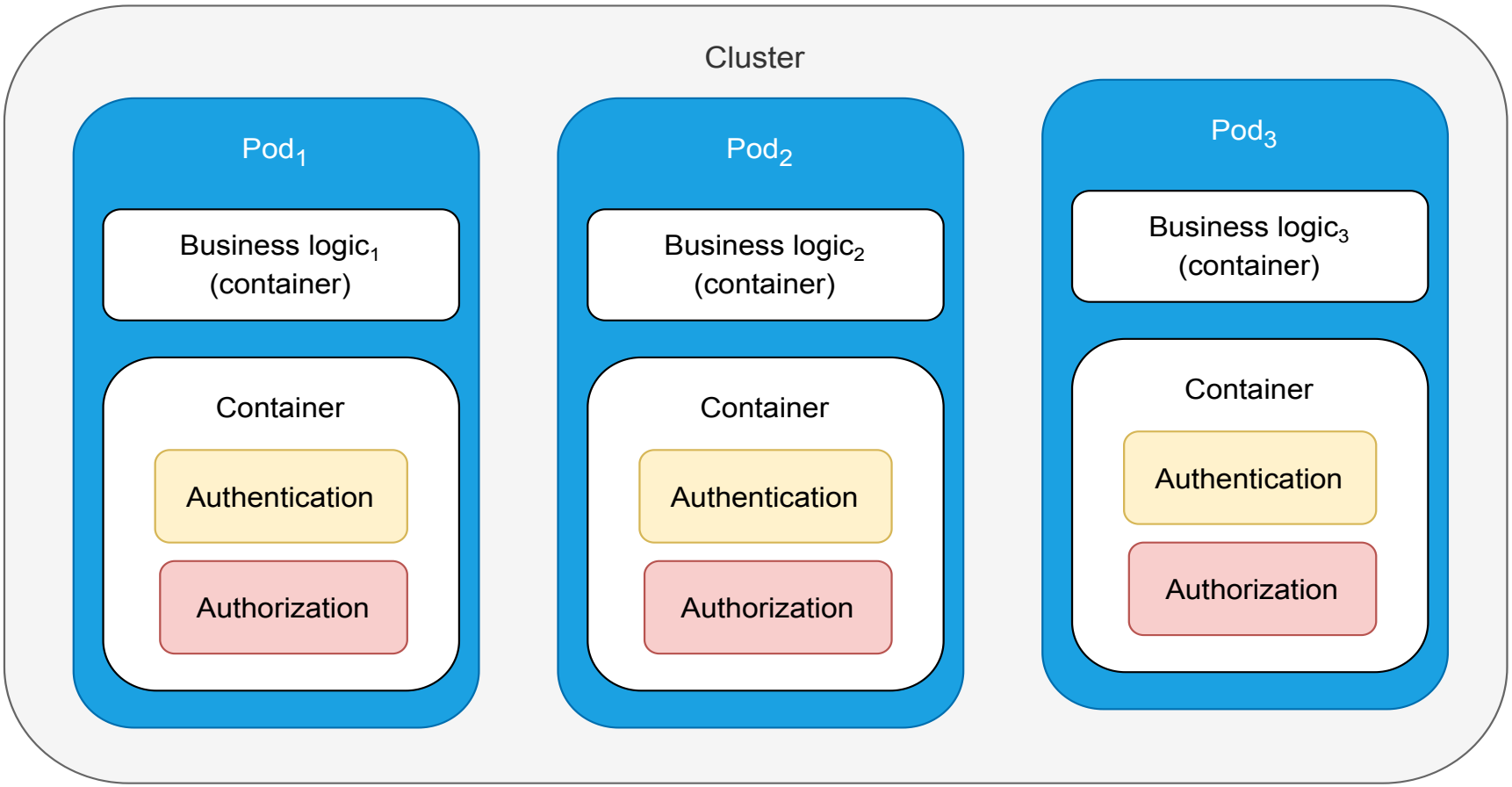


Figure 36: For Security, This Design Might Sometimes Be Better

Sidecars and the Zero-Trust Principle (3/5)

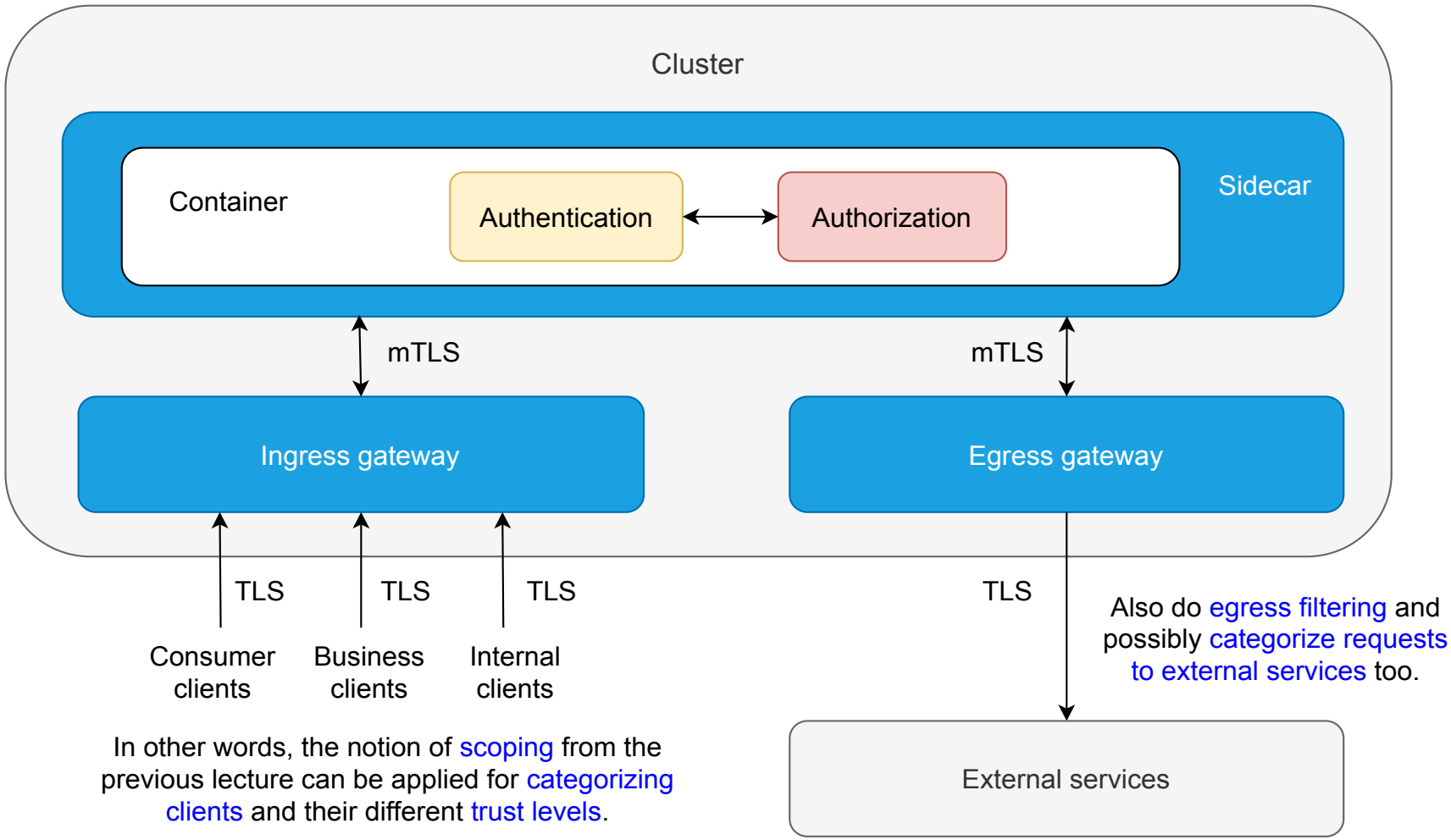


Figure 37: Also Output (Egress) Filtering Aligns With Sidecars

Sidecars and the Zero-Trust Principle (4/5)

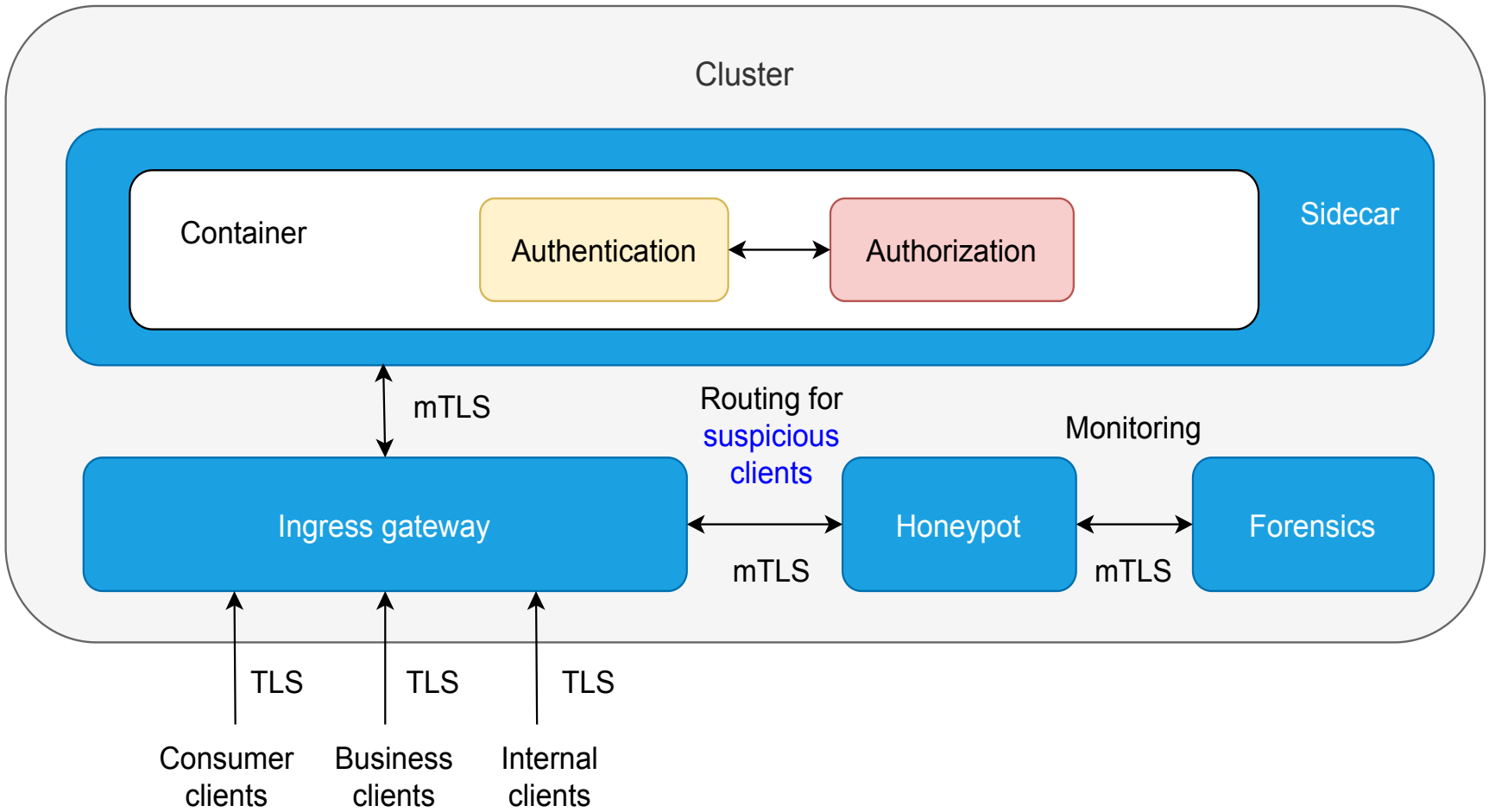


Figure 38: Cyber Security Is Also About Deception

Sidecars and the Zero-Trust Principle (5/5)

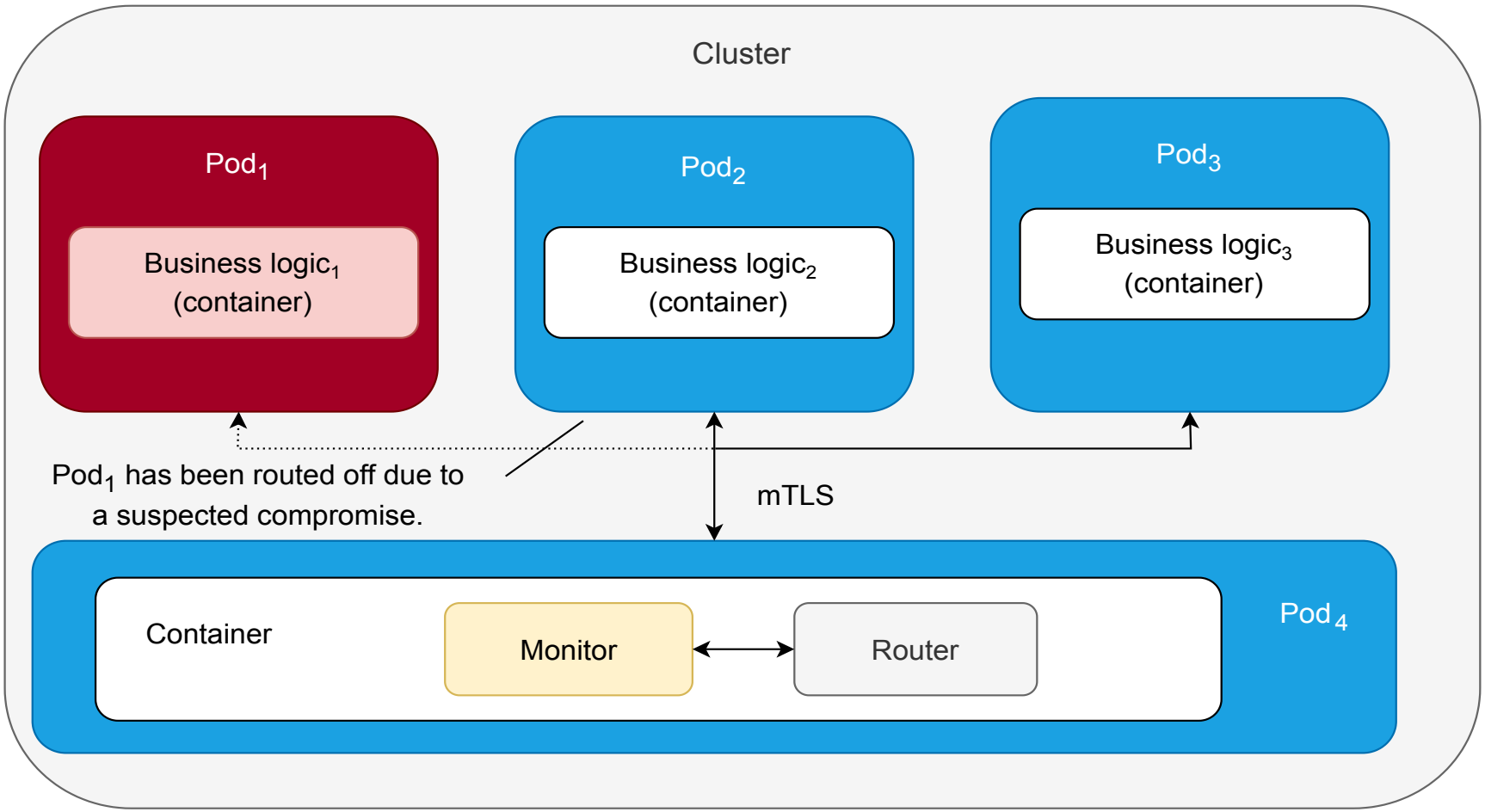


Figure 39: Isolation Is Important Also During Actual Incidents

On the Cyber Security Idiom of Confused Deputy

Figure 40: Source: Wikipedia (public domain)

Question

What Tactic from the Previous Lectures Could Be Perhaps Used for Preventing a Monitoring Component Ending as a Confused Deputy?

Recall Also the Complexity Trade-Off Once Again

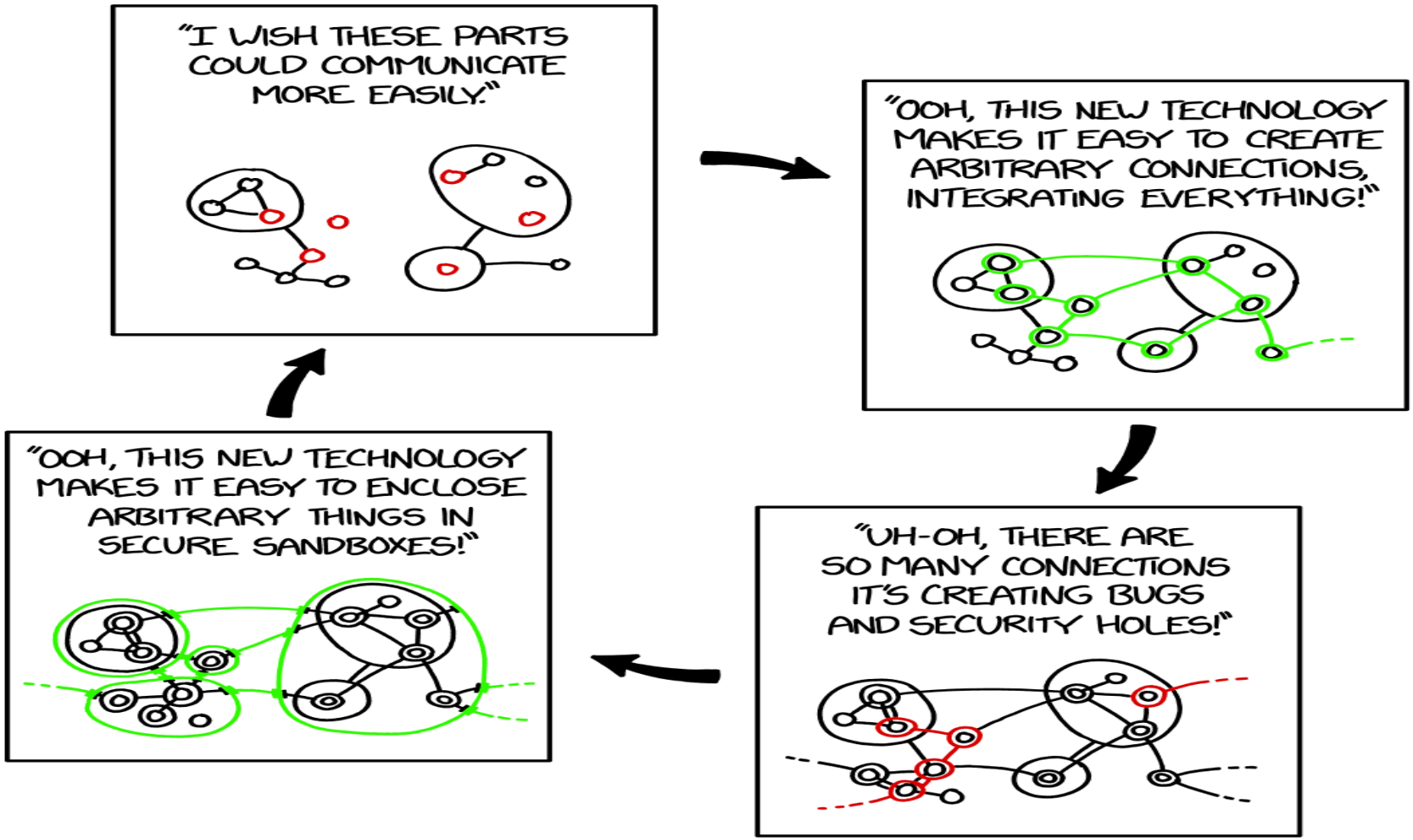


Figure 41: https://xkcd.com/2044/

Classifying Data (1/2)

- Thus, as the zero trust principle assumes a compromise by default, encryption at rest and egress filtering can generally reduce a blast radius in terms of data exfiltration

- In addition to isolation, segmentation, the zero trust and least privilege principles, and other principles already covered, Microsoft (2026b) recommends to **classify data** based on its type, sensitivity, and risks
  • Obviously, governments and state actors do such classification heavily but their practices generalize in general

Classifying Data (2/2)

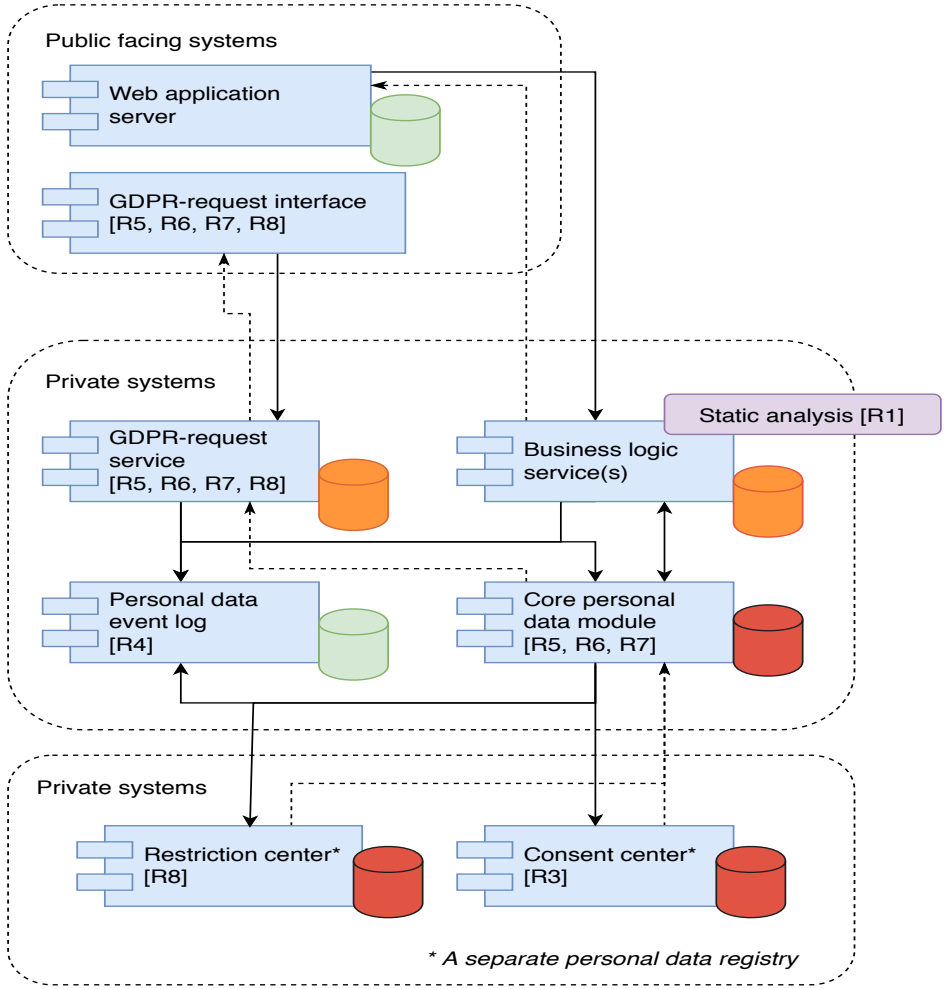


Figure 42: Personal Data Classification (Hjerppe 2019, Fig. 3)

Side-Channels (May) Affect Also Cloud Computing

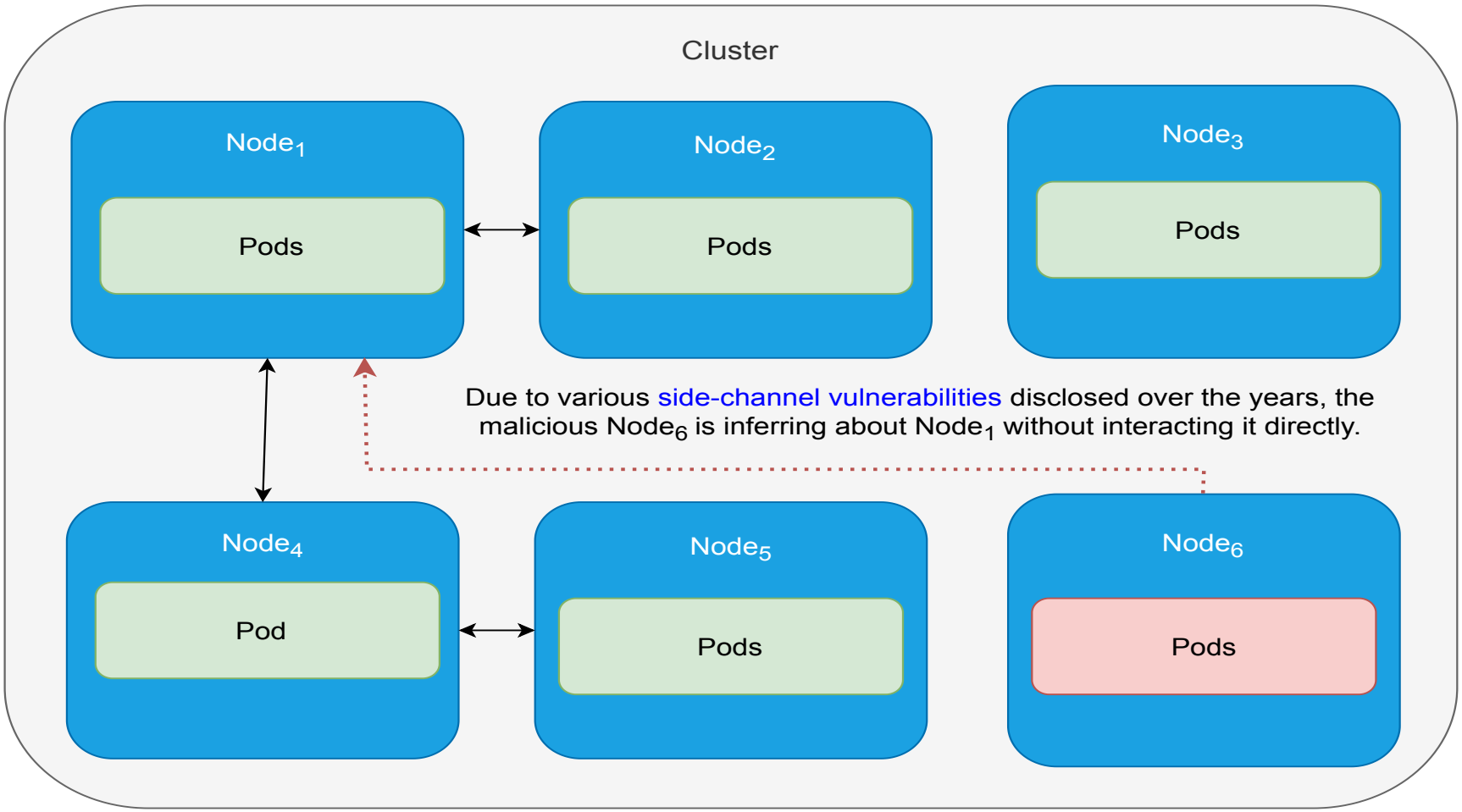


Figure 43: While It Remains Unclear How Realistic the Attack Is, the Potential Is There Due to Numerous Side-Channel Vulnerabilities

On Side-Channels More Generally

- While many side-channel attacks focus on bypassing the protection rings noted earlier (cf. CPUs, memory, caches, etc.), there are numerous other ways **hardware leaks**
  - Anything from timings and power consumption to acoustic noise and electromagnetic emissions
- It is difficult to defend against side-channel attacks, but in general, try to either **plug the leak** or **add noise**

Connectors $\mapsto$ Networking (1/7)

- Securing computer communications, data flows, and networking in general is obviously important and typically belongs to (preferably functional) security requirements
- A classical attack vector is **man-in-the-middle** (MitM) attacks, which can violate *confidentiality* (eavesdropping), *integrity* (intercepting and modifying data), and *availability* (intercepting and destroying or corrupting data)
- According Conti & Dragoni (2016), MitM attacks can be classified into those based on **impersonation**, given **communication channels**, and **locations** of attackers and targets in a network

Connectors $\mapsto$ Networking (2/7)

- Regarding communication channels, most of the Internet's core layers are theoretically a target

Table 1: Examples of Internet Protocols Against Which MitM Attacks Have Been Historically Conducted (Conti & Dragoni 2016, Table 1)

OSI layer	Protocols
7. Application	Border Gateway Protocol (BGP), Dynamic Host Configuration Protocol (DHCP), Domain Name System (DNS)
6. Presentation	Transport Layer Security (TLS)
4. Transport	Internet Protocol (IP)
2. Data link	Address Resolution Protocol (ARP)

Connectors $\mapsto$ Networking (3/7)

- Conti & Dragoni (2016) recommend the following seven design principles for countering MitM attacks:
  1. Use strong mutual authentication for all endpoints
  2. Exchange private keys using a secure channel
  3. Use few secure channels for integrity verification
  4. Sign public keys by a certified authority
  5. Use certificate pinning
  6. Encrypt communication
  7. Continuously examine behavior of interacting endpoints according to an agreed interaction protocol
- Of these principles, note that (6) is meaningless in case there are fatal flaws with (1) and (2), and maybe (4) too
- Note also that (7) essentially reiterates the earlier points about input/output validation at the networking context

Connectors ↦ Networking (4/7)

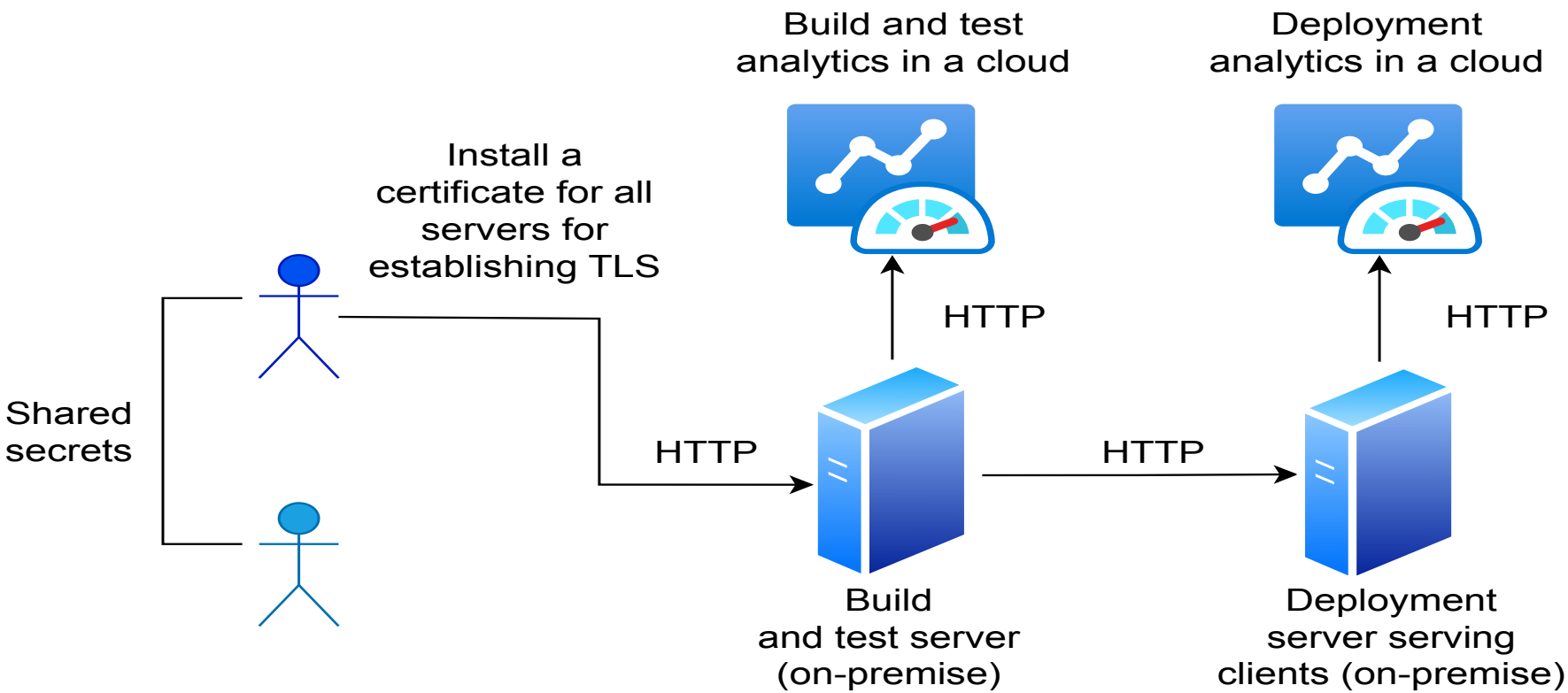


Figure 44: What Is Wrong Here?

Connectors ↦ Networking (5/7)

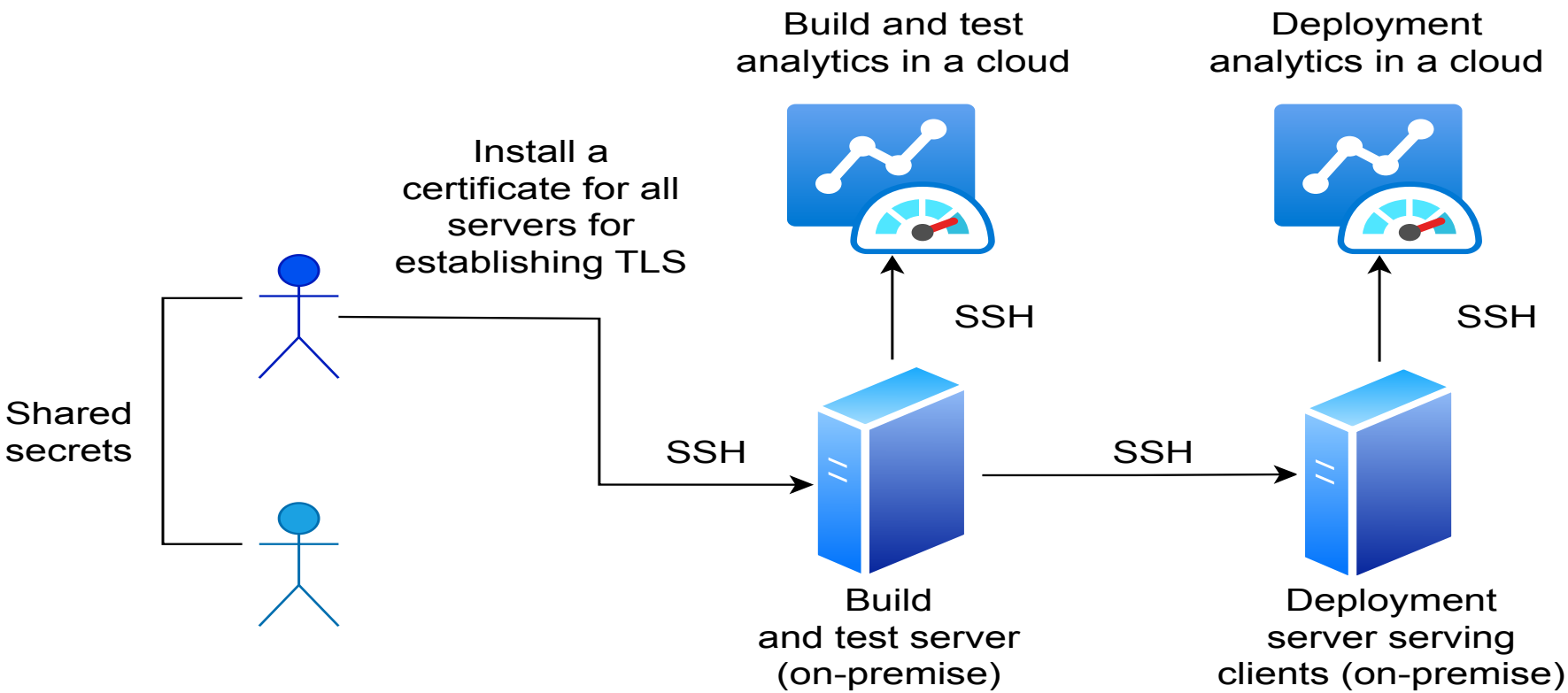


Figure 45: What Might Be Still Wrong Here?

Connectors ↦ Networking (6/7)

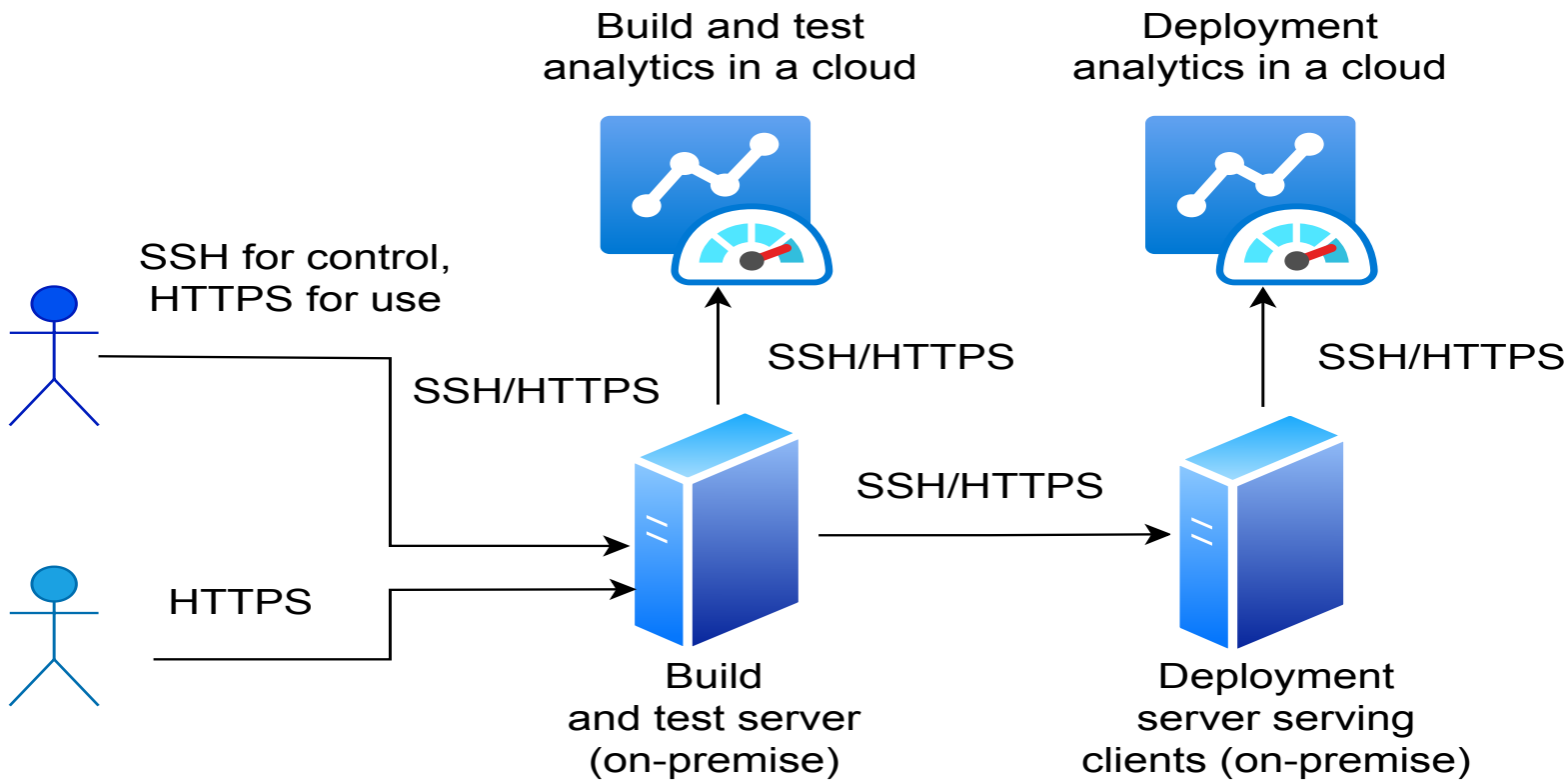


Figure 46: What Might Be Still Needed Here?

Connectors ↦ Networking (7/7)

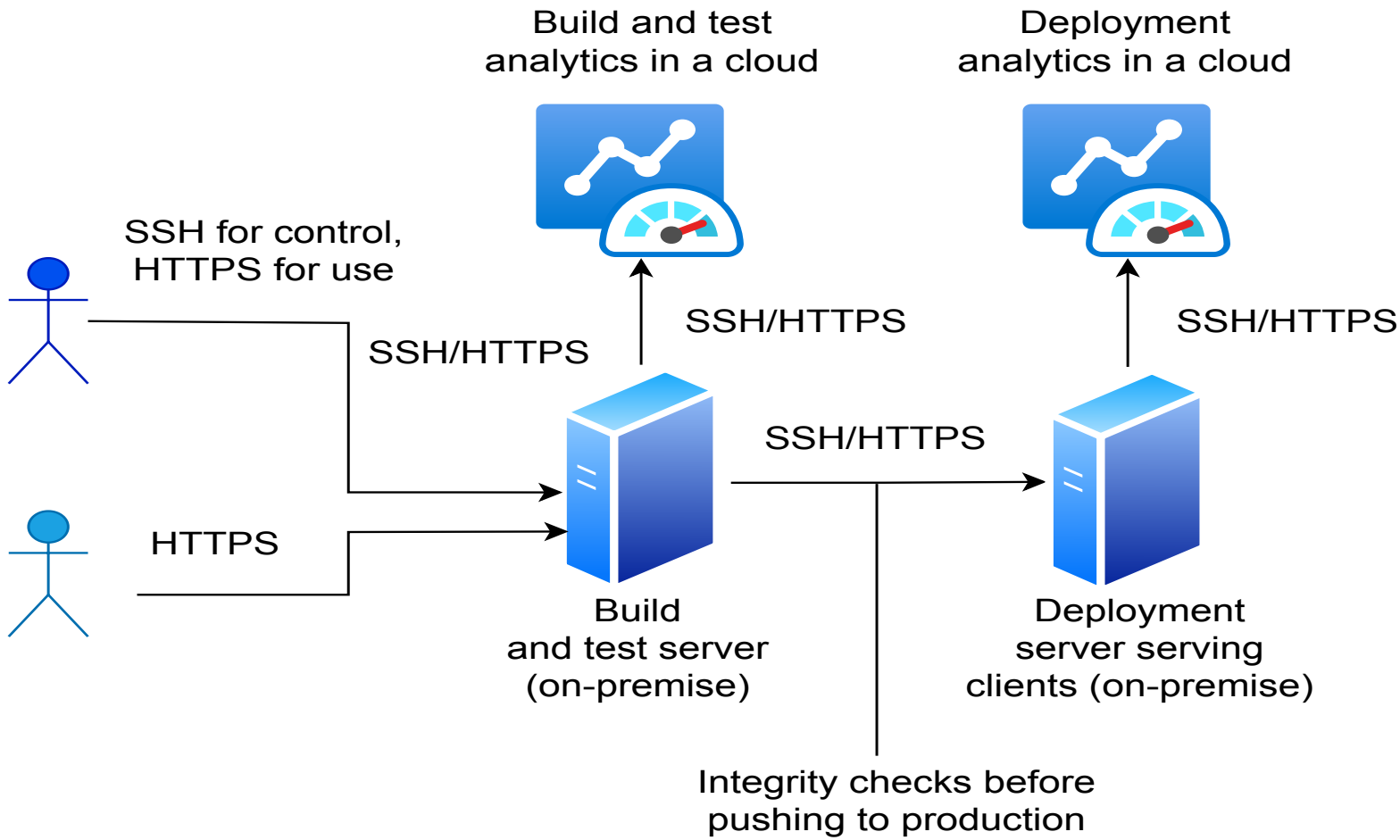


Figure 47: A Rather Typical Small CI/CD Setup

Finally, A Note on Cryptography

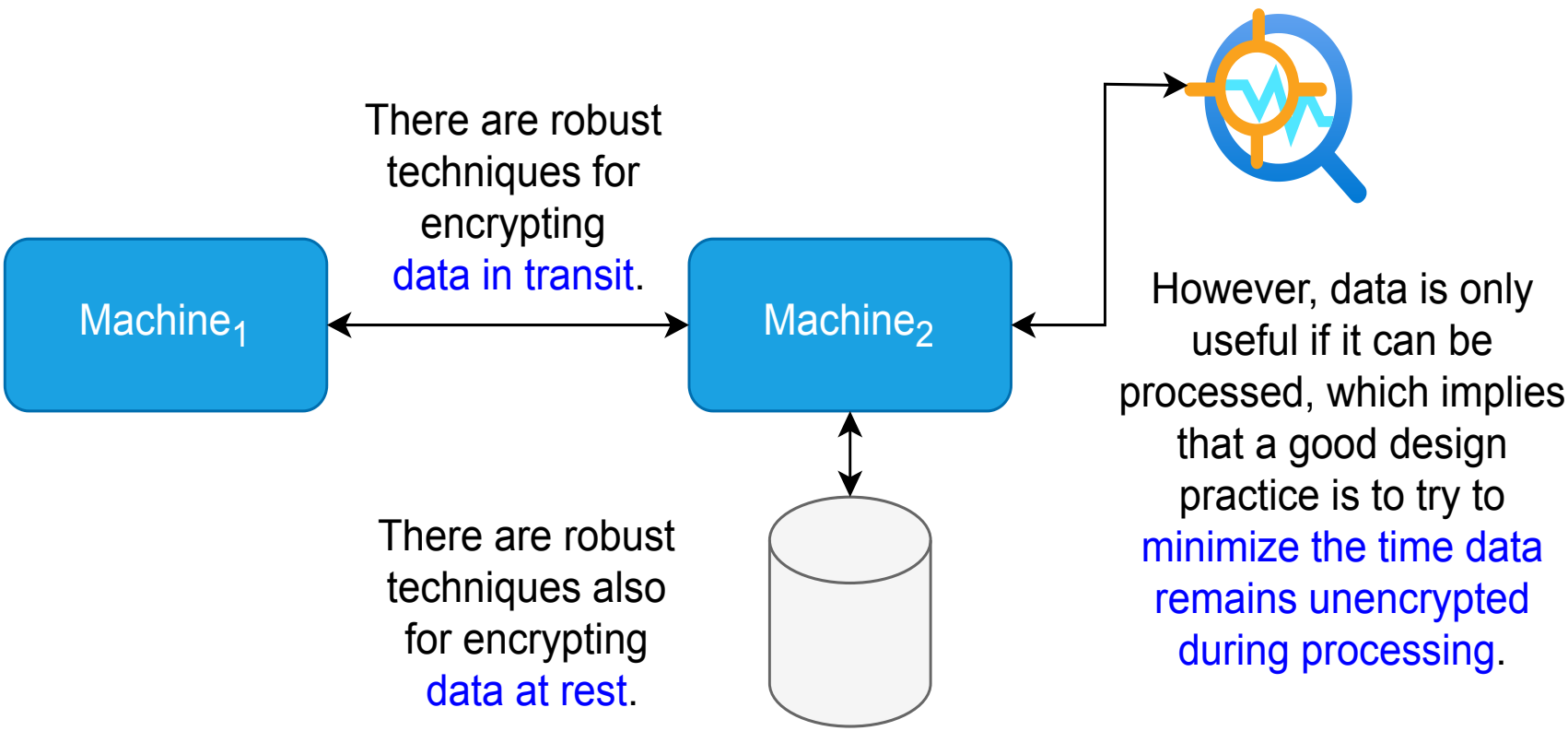


Figure 48: Designing for Encryption

Questions?

Additional References

1. Microsoft (2026a): Security Development Lifecycle (SDL) Practices. https://www.microsoft.com/en-us/securityengineering/sdl/practices
2. Zimmermann & Staicu & Pradel (2019): Small World with High Risks: A Study of Security Threats in the npm Ecosystem. USENIX Security, https://www.usenix.org/system/files/sec19-zimmermann.pdf
3. Microsoft (2026b): Security Design Principles. https://learn.microsoft.com/en-us/azure/well-architected/security/principles
4. NCSC (2025): Network Architectures. https://www.ncsc.gov.uk/collection/device-security-guidance/infrastructure/network-architectures
5. Hjerppe, K. & Ruohonen, J. & Leppänen, V. (2019): The General Data Protection Regulation: Requirements, Architectures, and Constraints. RE 2019, IEEE, https://arxiv.org/pdf/1907.07498
6. Conti, M. & Dragoni, N. (2016): A Survey of Man In The Middle Attacks. IEEE Communications Surveys & Tutorials 18(3), pp. 2027–2051, https://doi.org/10.1109/COMST.2016.2548426